



UNIVERSIDAD AUTÓNOMA METROPOLITANA

Unidad Azcapotzalco

División de Ciencias Básicas e Ingeniería
Licenciatura en Ingeniería en Computación



Sistema para la gestión de formularios sobre los cursos y autoevaluaciones docentes en la División de Ciencias y Artes para el Diseño (CyAD)

Proyecto Tecnológico

Reporte Final del Proyecto de Integración

Trimestre 2026-Primavera

Zuriel Godínez Ramos
2202003916
al2202003916@azc.uam.mx

M. en C Cesar Benavides Alvarez
Profesor Asociado
Departamento de Electrónica
cesarbenavidez@azc.uam.mx

M. en C Josué Figueroa González
Profesor Asociado
Departamento de Sistemas
jfgo@azc.uam.mx

21 de agosto de 2026

Declaratoria

Yo, César Benavides Álvarez, declaro que aprobé el contenido del presente Reporte de Proyecto de Integración y doy mi autorización para su publicación en la Biblioteca Digital, así como en el Repositorio Institucional de la UAM Azcapotzalco.

César Benavides Álvarez

Yo, Josué Figueroa González, declaro que aprobé el contenido del presente Reporte de Proyecto de Integración y doy mi autorización para su publicación en la Biblioteca Digital, así como en el Repositorio Institucional de la UAM Azcapotzalco.

Josué Figueroa González

Yo, Zuriel Godinez Ramos, doy mi autorización a la Coordinación de Servicios de Información de la Universidad Autónoma Metropolitana, Unidad Azcapotzalco, para publicar el presente documento en la Biblioteca Digital, así como en el Repositorio Institucional de la UAM Azcapotzalco. En caso de que el Comité de Estudios de la Licenciatura en Ingeniería en Computación apruebe la realización de la presente propuesta, otorgamos nuestra autorización para su publicación en la página de la División de Ciencias Básicas e Ingeniería.

Zuriel Godinez Ramos

Resumen

En la División de Ciencias y Artes para el Diseño (CyAD) de la Universidad Autónoma Metropolitana Unidad Azcapotzalco, los profesores entregan cada trimestre tres tipos de documentos: las *cartas temáticas* que describen el contenido de cada curso, los *requisitos de recuperación* para estudiantes en situación irregular y una *autoevaluación* única por trimestre de su propia labor docente. Hasta ahora este proceso se realizaba con formularios de Google y hojas de cálculo; esto implicaba tiempos muy largos para crear cada documento, dificultades para dar seguimiento y una alta probabilidad de errores, pues la mayoría de los datos eran ingresados manualmente.

Para atender esta necesidad, se desarrolló una aplicación web que permite a los profesores un manejo más sencillo de la información que deben capturar a lo largo del trimestre; como administrativo, se facilita el dar seguimiento a las actividades de los profesores y evaluar el cumplimiento por documento y por departamento.

La aplicación distingue tres tipos de usuario, cada uno con actividades bien delimitadas. El *administrador* inicia sesión con credenciales institucionales y se encarga de mantener actualizados los catálogos (departamentos, licenciaturas, posgrados, áreas y UEAs), dar de alta o de baja a los profesores, según se requiera, diseñar los formularios de autoevaluación (definiendo secciones, preguntas y rúbrica de puntajes), abrir y cerrar los periodos de captura para cada uno de los documentos, y consultar los reportes globales de cumplimiento que muestran qué profesores han entregado cada documento y cuáles siguen pendientes. El *docente* accede con sus credenciales para redactar, guardar como borrador y publicar sus propias cartas temáticas y requisitos de recuperación de cada UEA que imparte, responder la autoevaluación trimestral — cuyo nivel de desempeño se calcula automáticamente a partir de sus respuestas —, consultar el histórico de documentos que él mismo ha entregado en trimestres anteriores y despublicar un documento si necesita corregirlo. El *público en general* (estudiantes, aspirantes, personal académico externo) no requiere iniciar sesión: consulta libremente los documentos publicados a través de la vista pública, filtrando por trimestre, programa académico y UEA para localizar la información del curso o grupo que le interesa. La aplicación se organiza así alrededor de cinco funciones principales: acceso seguro con diferenciación de roles, captura y publicación de cartas temáticas, captura y publicación de requisitos de recuperación, llenado de autoevaluaciones con cálculo automático del nivel de desempeño, y generación de reportes de cumplimiento para la División.

En este reporte se describe el proceso de desarrollo, los resultados obtenidos, las diferencias entre lo comprometido y lo entregado.

Índice

Resumen	1
1. Introducción	8
2. Antecedentes	9
2.1. Trabajos relacionados	9
2.1.1. Google Forms en la evaluación diagnóstica como apoyo en las actividades docentes. Caso con estudiantes de la Licenciatura en Turismo [1].	9
2.1.2. La autoevaluación docente de aula: un camino para mejorar la práctica educativa [2].	9
2.1.3. Sistema para la configuración de formularios y captura de los resultados de la evaluación de Atributos de Egreso en un proceso de mejora continua [3].	9
2.1.4. Sistema académico de gestión escolar trimestralizado [4].	9
2.1.5. Sistema de Gestión Académica a través del desarrollo de Modelo-Vista-Controlador [5].	9
2.1.6. Sistema de Gestión Administrativa para la Escuela Secundaria No. 44 "Jorge Luis Borges"[6].	10
3. Justificación	12
4. Objetivos	13
4.1. Objetivo General	13
4.2. Objetivos Específicos	13
5. Marco teórico	14
5.1. Documentos académicos gestionados dentro del sistema	14
5.2. Glosario y siglas	16
5.3. Aplicación web y arquitectura cliente-servidor	19
5.4. Patrón Modelo-Vista-Controlador (MVC)	19
5.5. REST y APIs sobre HTTP	20
5.6. HTTP, <i>endpoints</i> , códigos de estado y CORS	20
5.7. Python y Django	20
5.8. Django REST Framework (DRF)	21
5.9. OpenAPI y <i>Swagger UI</i>	22
5.10. PostgreSQL	22
5.11. <i>Constraints</i> declarativos: unicidad compuesta, XOR y unicidad parcial	22
5.12. React	23
5.13. Vite	23
5.14. Tailwind CSS	23
5.15. Bibliotecas complementarias del frontend	24
5.16. Patrones frontend: SPA con <i>guards</i> , <i>interceptores</i> y estado de servidor	24

5.17. Componentes de interfaz: <i>form-builder</i> , cascada, acordeón, <i>dashboard</i> y <i>badge</i>	25
5.18. Autenticación con JSON Web Tokens (JWT)	25
5.19. Control de acceso basado en roles (RBAC)	26
5.20. Seguridad web y OWASP Top 10	26
5.21. Formatos de intercambio de datos	27
5.22. Máquina de estados y ciclo de vida de artefactos	27
5.23. Versionado inmutable y <i>snapshots</i> históricos	28
5.24. Escalas de medición: Likert y niveles de desempeño	28
5.25. Metodología, hitos y control de versiones	29
5.26. Pruebas automatizadas	29
6. Desarrollo del proyecto	30
6.1. Arquitectura general	30
6.1.1. Estructura de directorios	30
6.2. Metodología y evolución del desarrollo	31
6.3. Modelo de datos	32
6.4. Módulo 1: autenticación y gestión de usuarios	36
6.4.1. Autenticación con JWT	36
6.4.2. Creación de un profesor con acceso al sistema	37
6.4.3. Roles y permisos	38
6.5. Módulo 2: catálogos institucionales	38
6.5.1. Precarga inicial y <i>fixtures</i>	38
6.5.2. Importación masiva de UEAs desde CSV	39
6.5.3. Restricciones de integridad de UEA	39
6.6. Módulo 3: habilitación de periodos por recurso	40
6.6.1. Invariante “a lo sumo uno activo por recurso”	40
6.6.2. Interfaz de usuario	40
6.6.3. Efectos sobre otros módulos	40
6.6.4. Eliminación protegida	41
6.7. Módulo 4: cartas temáticas y requisitos de recuperación	41
6.7.1. Creación y publicación desde el rol profesor	41
6.7.2. Reglas de propiedad y ediciones fuera de tiempo	42
6.7.3. Vista del administrador	42
6.8. Módulo 5: <i>form-builder</i> y autoevaluación docente	42
6.8.1. Flujo operativo del administrador (<i>form-builder</i>)	42
6.8.2. Máquina de estados del formulario	43
6.8.3. Sistema de puntuación (<i>scoring</i> ponderado)	44
6.8.4. Privacidad de la ponderación mientras se contesta	45
6.8.5. Flujo operativo del profesor	45
6.9. Módulo 6: reportes de cumplimiento y descargas XLSX	46
6.9.1. Endpoints de agregación	46
6.9.2. Generación de XLSX del lado del cliente	46
6.10. Módulo 7: vista pública de exploración	47
6.10.1. Cascada de tres pasos	47
6.10.2. Vista imprimible	47

6.11. Aspectos transversales	48
6.11.1. Documentación viva de la API	48
6.11.2. Formato de error unificado	48
6.11.3. Pruebas automatizadas	48
6.12. Seguridad	49
7. Resultados	50
7.1. Métricas del código y de la base de datos	50
7.2. Métricas de pruebas	50
7.3. Evidencia visual del sistema en operación	51
7.3.1. Autenticación	51
7.3.2. Vista Docente	51
7.3.3. Vista Administrador	51
7.3.4. Vista pública	51
7.4. Documentación de la API	51
8. Análisis y discusión de resultados	52
8.1. Comparación entregado vs. propuesta	52
8.2. Discusión de las brechas frente a la propuesta	53
8.2.1. Identidad por email en lugar de número económico	53
8.2.2. Ausencia de exportación a PDF/DOCX en backend	53
8.2.3. Cartas y requisitos aplanados	54
8.2.4. Sin reset de contraseña autoservicio	54
8.3. Elementos entregados que no estaban en la propuesta	54
8.4. Limitaciones del sistema actual	54
9. Conclusiones	56
9.1. Cumplimiento por objetivo	56
9.2. Aprendizajes técnicos	56
9.3. Trabajo futuro	57
A. Código fuente	58
A.1. Estructura del repositorio	58
A.2. Backend — Modelo de Usuario con roles	58
A.3. Backend — Perfil de Profesor y Periodo	59
A.4. Backend — JWT y throttling	60
A.5. Backend — Documento academico base	61
A.6. Backend — Modelos del <i>form-builder</i>	62
A.7. Backend — Captura de respuestas y baremo	63
A.8. Backend — Scoring ponderado de autoevaluacion	65
A.9. Backend — Permisos por rol	66
A.10. Frontend — Cliente HTTP con refresh automatico de JWT	67
A.11. Frontend — Guard de rutas por rol	67
A.12. Frontend — Renderizador dinamico de preguntas	68
A.13. Backend — Comando de importacion CSV para UEA	68

A.14.Backend — Constraint XOR y unicidad parcial de UEA	69
A.15.Backend — Ciclo de vida de Formulario	69
A.16.Backend — Login con distincion de cuenta desactivada	70
A.17.Frontend — Generacion de XLSX por periodo	71
A.18.Nota sobre el listado completo	72
B. Entregables del plan de trabajo	72

Índice de figuras

1.	Arquitectura de tres capas del sistema.	31
2.	Diagrama de casos de uso del sistema.	77
3.	Diagrama Entidad-Relación del sistema (agrupado por app Django).	78
4.	Máquina de estados del <code>Formulario</code> de autoevaluación. Las transiciones que preservan la versión se dibujan a la izquierda entre <code>PUBLICADO</code> y <code>CERRADO</code> ; la revisión con incremento de versión (<code>publicar_revision</code>) se traza como arista externa.	78
5.	Pantalla de inicio de sesión.	79
6.	<i>Dashboard</i> del profesor: tarjetas de resumen y listas agrupadas por periodo.	79
7.	Lista de cartas temáticas del profesor.	80
8.	Formulario de creación/edición de carta temática.	80
9.	Formulario de requisitos de recuperación.	81
10.	Formulario de autoevaluación con renderizado dinámico de preguntas.	81
11.	<i>Dashboard</i> del administrador: KPIs globales y cumplimiento por departamento.	82
12.	Gestión de profesores: crear, editar, restablecer contraseña.	82
13.	Constructor de formularios de autoevaluación (<i>form-builder</i>).	83
14.	Panel de reportes con descarga en XLSX.	83
15.	Vista pública <i>Explorar</i> : cascada periodo→programa→UEA con acordeón de documentos.	84
16.	Vista pública de una carta temática publicada, con formato imprimible.	84

Índice de cuadros

1.	Comparación cualitativa de los trabajos relacionados con el proyecto propuesto.	11
2.	Métricas del código entregado.	50
3.	Volumen de pruebas por módulo.	51
4.	Grado de cumplimiento por componente respecto a la propuesta.	52
5.	Entregables prometidos en la propuesta y su estado real.	72

1. Introducción

En la Universidad Autónoma Metropolitana Unidad Azcapotzalco, particularmente en la División de Ciencias y Artes para el Diseño (CyAD), los profesores son responsables de llenar manualmente formularios en Google Forms que luego son capturados en hojas de cálculo para gestionar documentos clave como las cartas temáticas, las autoevaluaciones docentes y los requisitos de recuperación para alumnos. Este proceso, repetitivo y propenso a errores, dificulta la organización y el seguimiento adecuado de la información académica. Para resolver este problema, se propone el desarrollo de un sistema digital dirigido principalmente a los docentes, que automatice el llenado de datos académicos una vez que el usuario se haya identificado y seleccione el documento con el que va a trabajar. El sistema permitirá a los profesores: (1) gestionar cartas temáticas, (2) realizar su autoevaluación docente al finalizar el trimestre, y (3) subir requisitos de recuperación. Además, el sistema facilitará a los administradores el acceso a las actividades realizadas por los docentes y la generación de reportes sobre el cumplimiento de estas actividades.

2. Antecedentes

2.1. Trabajos relacionados

2.1.1. Google Forms en la evaluación diagnóstica como apoyo en las actividades docentes. Caso con estudiantes de la Licenciatura en Turismo [1].

Google Forms atiende la necesidad de recopilar, organizar y analizar información de manera eficiente. Permite crear encuestas y formularios sin conocimientos técnicos, con almacenamiento automático en Google Sheets e integración con otros servicios de Google. Además, permite utilizar plantillas prediseñadas o imágenes y logos propios.

2.1.2. La autoevaluación docente de aula: un camino para mejorar la práctica educativa [2].

El artículo aborda la temática de la autoevaluación docente como estrategia que permite recoger información relevante acerca del desempeño docente del aula. Se analiza la autoevaluación docente como una estrategia constructiva de mejora profesional. Se promueve la reflexión crítica sobre el desempeño en el aula, enfocándose en identificar fortalezas y áreas de oportunidad.

2.1.3. Sistema para la configuración de formularios y captura de los resultados de la evaluación de Atributos de Egreso en un proceso de mejora continua [3].

La evaluación de los Atributos de Egreso requiere información precisa en cada una de las evaluaciones. No obstante, al ser un proceso manual, es propenso a errores. El sistema automatiza la captura de información en procesos de evaluación académica. Incluye módulos que permiten precargar datos, validar usuarios y estructurar formularios según la UEA y los atributos a evaluar.

2.1.4. Sistema académico de gestión escolar trimestralizado [4].

Mediante un análisis preliminar realizado en la Unidad Educativa Técnico Humanístico "José Luis Suárez Guzmán" ubicada en la ciudad de El Alto, se observó que la institución no dispone de un sistema de monitoreo y registro virtual. El sistema propuesto centraliza la gestión académica, permitiendo consultar y registrar información pedagógica desde cualquier dispositivo. Mejora la trazabilidad y disponibilidad de los registros escolares.

2.1.5. Sistema de Gestión Académica a través del desarrollo de Modelo-Vista-Controlador [5].

La gestión administrativa suele manejar grandes volúmenes de datos, que sin el tratamiento correcto crean dificultades en las tareas. Este proyecto implementa módulos específicos para la gestión académica: planes de estudio, carga académica y disponibilidad docente. Utiliza una arquitectura Modelo-Vista-Controlador (MVC) para mejorar la organización del desarrollo.

2.1.6. Sistema de Gestión Administrativa para la Escuela Secundaria No. 44 "Jorge Luis Borges"[6].

El incremento significativo de la matrícula y del personal de la escuela produce un aumento en la cantidad de la documentación que se administra en forma manual, lo que origina errores en el registro de los datos. El sistema atiende el incremento de la carga administrativa, automatiza registros escolares y centraliza la información institucional. Fue construido con metodología XP, optimizando procesos en un entorno de cambio constante.

Las similitudes y diferencias con el proyecto propuesto se presentan en la Tabla 1.

Ref.	Similitudes	Diferencias
[1]	■ Uso de formularios digitales para capturar información. ■ No requiere conocimientos técnicos.	■ Permite trabajar individualmente o de forma colaborativa a distancia. ■ No tiene un sistema de autenticación con roles específicos.
[2]	■ Relevancia de la autoevaluación docente como herramienta de mejora. ■ Permitir que el docente identifique sus fortalezas y áreas de mejora.	■ Aborda la autoevaluación desde una perspectiva pedagógica, no tecnológica. ■ No plantea un sistema automatizado.
[3]	■ Captura estructurada de datos mediante formularios. ■ Autocompletado de formularios a partir de la información en una base de datos.	■ Enfocado en evaluar atributos de egreso por UEA. ■ Módulo para indicar los atributos de egreso a evaluar. ■ Módulo para asignar a un profesor a la UEA a evaluar.
[4]	■ Registro y visualización de información académica.	■ Orientado al seguimiento pedagógico de estudiantes, no a documentos docentes.
[5]	■ Módulos especializados para distintos procesos académicos. ■ Manejo de información de los planes de estudios.	■ Gestión enfocada en planes de estudio y disponibilidad docente, no en formularios.
[6]	■ Centralización de datos. ■ Automatización de tareas administrativas.	■ Diseñado principalmente para el personal administrativo, no para profesores.

Ref.	Similitudes	Diferencias
-------------	--------------------	--------------------

Tabla 1. Comparación cualitativa de los trabajos relacionados con el proyecto propuesto.

3. Justificación

El principal beneficio de este sistema es la optimización del tiempo, permitiendo un manejo más sencillo de la información que deben capturar los profesores. Además, esta solución contribuye a la modernización de la administración académica, alineándose con la tendencia de digitalización en las instituciones educativas [7]. Un sistema bien estructurado no solo beneficiará a los docentes, sino que también permitirá a la administración académica tener un mejor control y seguimiento de la información generada, promoviendo una gestión más eficiente y transparente.

4. Objetivos

4.1. Objetivo General

Desarrollar un sistema que permita a los profesores de la división de CyAD gestionar de manera eficiente sus datos académicos, la información sobre sus cursos impartidos, los requisitos de sus evaluaciones de recuperación, y su autoevaluación al final de cada trimestre.

4.2. Objetivos Específicos

1. Permitir el acceso al sistema mediante una autenticación que permita la identificación y uso del sistema según el rol del usuario.
2. Crear, modificar y consultar información sobre cartas temáticas y evaluaciones de recuperación por parte de los profesores.
3. Registrar información sobre la autoevaluación de los profesores al finalizar cada trimestre.
4. Crear reportes que faciliten al personal administrativo el seguimiento del cumplimiento de las actividades docentes.

5. Marco teórico

En esta sección se describen detalladamente los conceptos, tecnologías y herramientas necesarias para llevar a cabo la implementación del proyecto. Inicialmente se desglosan las definiciones relacionadas con los documentos académicos gestionados dentro del sistema y palabras clave utilizadas a lo largo del documento; posteriormente, se definen conceptos relacionados con las tecnologías utilizadas para el desarrollo del sistema.

5.1. Documentos académicos gestionados dentro del sistema

El sistema gira alrededor de dos documentos institucionales que cada profesor elabora por cada UEA que imparte a lo largo de un trimestre: la *carta temática* y los *requisitos de recuperación*. Ambos se depositan en un espacio público consultable por alumnos y personal académico y comparten un conjunto de metadatos comunes: profesor autor, UEA, periodo trimestral, nombre e identificador de grupo, horario, modalidad (presencial, remoto o mixto) y estado del documento (*borrador* mientras se redacta, *publicado* una vez difundido).

Carta temática

Una *carta temática* es el programa detallado con el que un profesor comunica cómo va a impartir una UEA durante un trimestre. No sustituye al programa oficial de la licenciatura o del posgrado — que es fijo y aprobado por la División —, sino que lo particulariza: bajo qué enfoque se desarrollará el curso, con qué calendario, con qué materiales y bajo qué criterios de evaluación.

En el sistema, cada carta temática se compone de los siguientes campos de contenido, redactados como texto libre por el profesor:

Descripción de la UEA Presentación general de la asignatura y su ubicación dentro del plan de estudios.

Objetivo general Propósito global que se espera que el estudiante alcance al concluir el trimestre.

Objetivos particulares Metas específicas que, en conjunto, permiten alcanzar el objetivo general.

Contenido sintético Enumeración de los temas y subtemas que se abordarán.

Objetivos de aprendizaje Resultados observables (conocimientos, habilidades, actitudes) que el estudiante deberá demostrar.

Requerimientos Materiales, herramientas, equipo o *software* que el estudiante necesita para cursar la UEA.

Conocimientos previos Contenidos que el estudiante debe dominar antes de iniciar el curso.

Modalidad de evaluación Instrumentos y ponderaciones con los que se calificará al estudiante.

Revisiones y asesorías Momentos y modos en que el profesor atiende dudas o revisa avances.

Bibliografía Fuentes básicas y complementarias recomendadas.

Enlace Liga(s) al aula virtual, videoconferencia o repositorio, cuando la modalidad lo requiere.

Calendarización de actividades Secuencia semana a semana de temas, entregas y evaluaciones.

Requisitos de recuperación

Los *requisitos de recuperación* constituyen el documento con el que el profesor comunica en qué consiste la *evaluación global* que la normatividad de la UAM ofrece a los estudiantes que no aprueban la UEA por la vía ordinaria. Su publicación oportuna es indispensable porque el estudiante debe conocer, con antelación, cuáles son las condiciones exactas para presentarla: dónde, cuándo, con qué materiales y bajo qué reglas.

En el sistema, cada requisito de recuperación se compone de los siguientes campos:

Lugar Espacio físico o virtual donde se presentará la evaluación. Si es remoto, puede incluir liga y contraseña.

Duración aproximada Tiempo estimado que tomará la evaluación.

Fecha y hora Momento en que se aplicará la evaluación, como texto libre (por ejemplo, "Jueves 03 de septiembre, 10:00 h").

Recursos necesarios Materiales, herramientas o equipo que el estudiante debe traer o tener disponibles.

Requisitos Condiciones que el estudiante debe cumplir para poder presentar la evaluación (por ejemplo, entrega previa de investigación, maqueta o porcentaje mínimo de tareas del trimestre).

Notas Aclaraciones adicionales que el profesor considere pertinentes.

Los campos son de texto libre y no obligatorios individualmente, para acomodar la heterogeneidad de las UEA de la División (talleres, laboratorios, seminarios teóricos, cursos híbridos), pero se validan como conjunto al momento de publicar. Ambos documentos son únicos por la combinación (*profesor, periodo, UEA, grupo*): un mismo profesor no puede tener dos cartas temáticas ni dos requisitos vigentes para el mismo grupo en un mismo trimestre.

5.2. Glosario y siglas

Este apartado reúne, como referencia rápida, la terminología institucional UAM-A/CyAD, las siglas técnicas y las bibliotecas mencionadas a lo largo del reporte.

Terminología institucional UAM-A / CyAD

UAM Universidad Autónoma Metropolitana

División Nivel organizativo intermedio entre la Unidad y los Departamentos. En este proyecto: División de Ciencias y Artes para el Diseño (CyAD).

Departamento Unidad organizativa académica que agrupa profesores por área disciplinar. CyAD tiene cuatro: *Evaluación del Diseño en el Tiempo, Investigación y Conocimiento para el Diseño, Procesos y Técnicas de Realización y Medio Ambiente*.

Licenciatura Programa de estudios de pregrado (ej. Arquitectura, Diseño de la Comunicación Gráfica, Diseño Industrial).

Posgrado Programa de estudios de nivel superior a la licenciatura.

Coordinación de Estudios Instancia académica-administrativa responsable de la operación de un programa docente.

Plan de estudios Documento oficial que fija las UEAs, su secuencia y sus créditos para una licenciatura o posgrado.

UEA *Unidad de Enseñanza-Aprendizaje*; corresponde a una asignatura en otros sistemas académicos. Se identifica por una clave numérica (ej. 1132049).

Área de conocimiento Agrupación temática de las UEAs dentro de un programa (por ejemplo “Composición”, “Cálculo estructural”). Distinta del Departamento.

Trimestre Unidad temporal académica de la UAM. Un plan de estudios se cursa nominalmente en 12 trimestres.

Periodo YY-{I,P,O} Nomenclatura del trimestre en el sistema: YY son los dos últimos dígitos del año y la letra codifica Invierno, Primavera u Otoño, respectivamente. Por ejemplo, 26-P designa el trimestre de Primavera de 2026.

Modalidad Forma en que se imparte una UEA: presencial, remoto o mixto.

Evaluación global Evaluación única prevista por la normatividad UAM para estudiantes que no aprobaron por vía ordinaria.

Número económico Identificador numérico único que la UAM asigna a cada trabajador académico.

Siglas técnicas

ACID *Atomicity, Consistency, Isolation, Durability* — propiedades que garantizan la corrección de las transacciones en un RDBMS.

API *Application Programming Interface* — contrato de operaciones que un componente expone a otros.

CI/CD *Continuous Integration / Continuous Delivery* — automatización de compilación, pruebas y despliegue tras cada cambio versionado.

CORS *Cross-Origin Resource Sharing* — protocolo del navegador que autoriza peticiones entre orígenes distintos (ver § 5.6).

CRUD *Create, Read, Update, Delete* — las cuatro operaciones básicas sobre un recurso persistente.

CSRF *Cross-Site Request Forgery* — ataque en el que un sitio malicioso induce al navegador de una víctima a enviar peticiones autenticadas a otro sitio.

DOM *Document Object Model* — representación en árbol de un documento HTML manipulable por JavaScript.

E2E *End-to-end* — prueba que ejercita el sistema completo desde la perspectiva del usuario.

FK *Foreign Key* — clave foránea; columna que referencia la clave primaria de otra tabla.

HMAC *Hash-based Message Authentication Code* — construcción que combina una función *hash* (SHA-256 en este proyecto) con una clave secreta para firmar mensajes.

HMR *Hot Module Replacement* — técnica de Vite que reemplaza módulos en el navegador sin recargar la página.

HTTP / HTTPS *HyperText Transfer Protocol* (y su variante cifrada con TLS) — protocolo de aplicación de la web.

JWT *JSON Web Token* — token firmado usado como credencial de sesión sin estado (RFC 7519).

KPI *Key Performance Indicator* — indicador clave de desempeño mostrado en los *dashboards*.

LDAP *Lightweight Directory Access Protocol* — protocolo estándar de directorios institucionales de usuarios.

ORM *Object-Relational Mapper* — capa que traduce entre objetos y filas de una base de datos relacional.

PBKDF2 *Password-Based Key Derivation Function 2* — algoritmo estándar para derivar una huella robusta a partir de una contraseña.

PDF *Portable Document Format* — formato de documento paginado listo para imprimir.

DOCX Formato de documento de Microsoft Word.

RBAC *Role-Based Access Control* — control de acceso basado en roles.

RDBMS *Relational Database Management System* — gestor de base de datos relacional (PostgreSQL en este proyecto).

REST *Representational State Transfer* — estilo arquitectónico para APIs sobre HTTP.

SMTP *Simple Mail Transfer Protocol* — protocolo estándar de envío de correo electrónico.

SPA *Single Page Application* — aplicación web cuya navegación ocurre sin recargar la página.

SSO *Single Sign-On* — mecanismo de inicio de sesión unificado entre varios sistemas.

SQL *Structured Query Language* — lenguaje estándar de consulta relacional.

TLS *Transport Layer Security* — protocolo que cifra el canal HTTP en HTTPS.

UI / UX *User Interface / User Experience* — interfaz de usuario / experiencia de usuario.

URI / URL *Uniform Resource Identifier / Locator* — identificador (y localizador) de un recurso en la web.

WSGI *Web Server Gateway Interface* — especificación de interfaz entre servidores web y aplicaciones Python.

XLSX Formato de libro de Microsoft Excel.

XSS *Cross-Site Scripting* — inyección de script malicioso en una página vista por otro usuario.

YAML *YAML Ain't Markup Language* — formato jerárquico textual usado por el esquema OpenAPI.

Bibliotecas y herramientas mencionadas

SheetJS (xlsx) Genera libros XLSX en el navegador; se usa para descargar reportes desde el rol administrador.

drf-spectacular Genera la especificación OpenAPI y la interfaz Swagger UI desde el código de DRF.

django-cors-headers Middleware que gestiona la lista blanca de orígenes CORS.

django-filter Añade filtros declarativos por *query string* a los *ViewSet*s de DRF.

python-decouple Lee la configuración desde archivos `.env`.

psycopg2 Adaptador PostgreSQL para Python que usa el ORM de Django.

djangoestframework-simplejwt Implementación de JWT (*access* + *refresh* con rotación) para DRF.

factory-boy Construcción de instancias de prueba con datos consistentes.

coverage.py* y *pytest-cov Miden la cobertura de código ejecutado por las pruebas.

pytest-django Integración de *pytest* con Django (fixtures de BD, cliente de pruebas, aislamiento).

WeasyPrint* y *ReportLab Generadores de PDF del lado del servidor considerados y descartados por sus dependencias nativas (GTK/Cairo).

Gunicorn* y *Nginx *Application server* WSGI y servidor HTTP inverso recomendados para el despliegue productivo (fuera del alcance actual).

Vitest*, *Jest*, *React Testing Library Herramientas típicas para probar componentes React; se mencionan como trabajo futuro (el proyecto aún no las incorpora).

Lighthouse* y *axe-core Herramientas de auditoría de rendimiento y accesibilidad, mencionadas como línea de trabajo pendiente.

Google Forms* / *Google Sheets Combinación que la Coordinación usaba antes del sistema (fuente del formulario ad hoc de autoevaluación y de la hoja de cálculo con los resultados); se referencia como motivación del proyecto.

5.3. Aplicación web y arquitectura cliente–servidor

Una *aplicación web* es un programa cuya interfaz se ejecuta dentro de un navegador y cuya lógica principal reside en un servidor remoto. A diferencia de una aplicación de escritorio, no requiere instalación: basta con una URL. La arquitectura habitual es *cliente–servidor*, en la que dos programas independientes se comunican a través de una red: el *cliente* solicita datos u operaciones y el *servidor* las procesa y responde.

5.4. Patrón Modelo–Vista–Controlador (MVC)

El *Modelo–Vista–Controlador* es un patrón arquitectónico introducido por Krasner y Pope para Smalltalk-80 [8]. Divide una aplicación con interfaz gráfica en tres responsabilidades:

- **Modelo:** representa los datos y las reglas de negocio. Es independiente de cómo se muestran o se manipulan.
- **Vista:** presenta el estado del modelo al usuario. Puede haber varias vistas de un mismo modelo.
- **Controlador:** recibe las entradas del usuario, decide qué operación realizar sobre el modelo y actualiza la vista.

5.5. REST y APIs sobre HTTP

REST (Representational State Transfer) es un estilo arquitectónico definido por Roy Fielding en su tesis doctoral [9]. Establece un conjunto de restricciones para diseñar servicios distribuidos escalables sobre HTTP:

- Cada recurso se identifica por una URI única (por ejemplo, `/api/v1/cartas-tematicas/42/`).
- Las operaciones se expresan con los verbos HTTP estándar: GET (leer), POST (crear), PUT/PATCH (actualizar), DELETE (eliminar). Al conjunto de estas cuatro operaciones se le conoce como *CRUD*.
- Las respuestas usan una representación acordada (habitualmente JSON) y son *sin estado*: cada petición contiene toda la información necesaria para procesarla.

5.6. HTTP, endpoints, códigos de estado y CORS

HTTP (HyperText Transfer Protocol) es el protocolo de aplicación de la web sobre el que se transporta la API. Su variante segura, *HTTPS*, cifra el canal con TLS y se recomienda en producción. Cada intercambio HTTP consta de una petición (verbo, URI, cabeceras, cuerpo opcional) y una respuesta (código de estado, cabeceras, cuerpo opcional).

Se llama *endpoint* a cada URI concreta que el servidor expone para un recurso u operación — por ejemplo, `POST /api/v1/auth/login/`. En este reporte los términos “endpoint” y “ruta de la API” se usan como sinónimos, siempre entendidos como el par (verbo, URI) que el cliente invoca.

Los *códigos de estado* más relevantes para el sistema son: 200 OK (éxito con contenido), 201 Created (recurso creado), 204 No Content (éxito sin cuerpo), 400 Bad Request (payload inválido), 401 Unauthorized (token ausente o expirado), 403 Forbidden (autenticado pero sin permiso), 404 Not Found, 409 Conflict (violación de unicidad) y 429 Too Many Requests (*rate limit* excedido; devuelto por el *throttling* del endpoint de login).

CORS (Cross-Origin Resource Sharing) es el mecanismo con el que un navegador permite que un documento cargado desde un origen (esquema + dominio + puerto) haga peticiones a otro origen distinto. Por defecto, la *política del mismo origen* lo prohíbe; el servidor debe declarar explícitamente qué orígenes acepta mediante cabeceras `Access-Control-Allow-Origin` y responder a las peticiones de *preflight* (OPTIONS). En este proyecto el backend mantiene una lista blanca de orígenes autorizados (por ejemplo, el frontend en `http://localhost:5173` durante el desarrollo) configurada vía la variable `CORS_ALLOWED_ORIGINS` y aplicada por la biblioteca *django-cors-headers*.

5.7. Python y Django

Python es un lenguaje de programación de alto nivel, interpretado y de tipado dinámico, ampliamente utilizado en desarrollo web.

Django [10] es un *framework* web escrito en Python que sigue el principio “*baterías incluidas*”: provee, listos para usar, los componentes que la mayoría de las aplicaciones web necesitan. Los más relevantes para este proyecto son:

- **ORM (*Object-Relational Mapper*)**: permite definir el esquema de la base de datos como clases de Python (los modelos) y consultarla con métodos en lugar de SQL. Reduce errores de sintaxis, previene inyección SQL y hace portable el código entre motores de base de datos.
- **Sistema de migraciones**: cada cambio al modelo se traduce automáticamente en un archivo de migración versionable en git. Permite evolucionar la base de datos de forma reproducible entre entornos.
- **Sistema de autenticación**: usuarios, grupos, permisos y *hashing* de contraseñas ya vienen implementados; el proyecto extiende el modelo de usuario para agregar rol.
- **Panel de administración**: interfaz autogenerada para operar la base de datos, útil como respaldo del administrador.
- **Sistema de validación y formularios**: encapsula las reglas de negocio en el servidor.

5.8. Django REST Framework (DRF)

Django REST Framework [11] es una capa sobre Django que facilita construir APIs REST. Sus componentes principales son:

- ***Serializers***: traducen entre objetos de Python (instancias de modelos) y representaciones serializables (JSON). Validan la entrada y controlan qué campos se exponen hacia afuera.
- ***ViewSet* y *routers***: agrupan las operaciones CRUD sobre un recurso y las publican bajo URLs consistentes con las convenciones REST.
- **Permisos (*permission classes*)**: clases reutilizables que declaran quién puede acceder a cada acción (por ejemplo, “sólo el dueño del recurso puede modificarlo”). Se aplican a nivel de *ViewSet* y, mediante `has_object_permission`, también a nivel de instancia.
- ***Throttling* y *rate limiting***: mecanismos equivalentes (el segundo es el término genérico) que limitan el número de peticiones que un cliente puede hacer en un lapso; útiles para mitigar ataques de fuerza bruta.
- ***drf-spectacular***: complemento que analiza el código y genera automáticamente la especificación *OpenAPI* de la API, junto con un explorador *Swagger UI* interactivo.

En el proyecto DRF permite mantener la definición del contrato de la API cercana al modelo, con *serializers* explícitos que documentan qué campos son públicos y qué reglas de validación aplican.

5.9. OpenAPI y Swagger UI

OpenAPI es una especificación abierta para describir contratos de APIs REST en un formato máquina-legible (JSON o YAML). Un documento OpenAPI enumera los *endpoints*, los verbos que aceptan, los esquemas de los cuerpos de petición y respuesta, los códigos de estado posibles y los mecanismos de autenticación. Sirve como documentación viva, como insumo para generadores de clientes en distintos lenguajes y como fuente para pruebas de contrato.

Swagger UI es una interfaz web que consume un documento OpenAPI y produce un explorador navegable en el que se pueden probar las peticiones interactivamente. En el proyecto, *drf-spectacular* genera la especificación desde las anotaciones del código de DRF y la expone en dos rutas: `/api/schema/` (documento OpenAPI en YAML descargable) y `/api/docs/` (Swagger UI). Con esto se evita la desincronización clásica entre la documentación redactada a mano y el comportamiento real del servidor.

5.10. PostgreSQL

PostgreSQL [12] es un sistema de gestión de bases de datos relacional (RDBMS), libre y de código abierto, con más de 30 años de desarrollo. Se seleccionó principalmente por su compatibilidad con el ORM de Django.

5.11. Constraints declarativos: unicidad compuesta, XOR y unicidad parcial

Una *restricción declarativa* (*constraint*) se define a nivel del esquema de la base de datos y es el motor quien la hace cumplir. Complementa la validación aplicativa (en *serializers* y modelos) como *defensa en profundidad*: aunque una capa falle, la otra sigue previniendo estados inconsistentes. El proyecto usa tres variantes:

- **Unicidad compuesta:** `UniqueConstraint(fields=["profesor", "periodo", "uea", "id_grupo"])` garantiza que un profesor no tenga dos cartas temáticas para el mismo grupo en el mismo trimestre para la misma UEA. La validación se realiza en el motor con un índice único; el intento de duplicación devuelve un error de integridad que DRF traduce a 400 Bad Request.
- **Restricción XOR:** expresa “exactamente uno de un conjunto de campos debe estar presente”. En el proyecto se aplica a UEA, que debe apuntar a una licenciatura o a un posgrado, nunca a ambos ni a ninguno. Se implementa como `CheckConstraint` con predicados `Q(licenciatura__isnull=False) & Q(posgrado__isnull=True)` en disyunción con su opuesto.
- **Unicidad parcial:** `UniqueConstraint(condition=...)` restringe la unicidad a un subconjunto de filas. Se usa para forzar clave única por programa: la misma clave textual (por ejemplo “11XY99”) puede coexistir en una licenciatura y en un posgrado, pero no repetirse dentro del mismo programa.

5.12. React

React [13] es una biblioteca de JavaScript, desarrollada originalmente por Facebook (hoy Meta), para construir interfaces de usuario a partir de *componentes* reutilizables.

Los conceptos centrales de React que se aprovechan son:

- **Componente:** función que recibe *props* (parámetros) y devuelve una descripción declarativa de la interfaz. Los componentes se componen entre sí formando árboles.
- **Estado (*state*):** datos internos de un componente que, al cambiar, disparan un nuevo renderizado.
- **Hooks:** funciones especiales (*useState*, *useEffect*, *useContext*, etc.) que permiten manejar estado y efectos secundarios en componentes funcionales.
- **Renderizado declarativo:** el programador describe cómo se ve la interfaz en función del estado; React se encarga de calcular y aplicar los cambios mínimos al DOM.

5.13. Vite

Vite [14] es una herramienta de *build* y servidor de desarrollo para aplicaciones frontend modernas que aprovecha los módulos nativos de JavaScript (*ES modules*) del navegador para arrancar el servidor en milisegundos, incluso en proyectos grandes.

Sus aportes en este proyecto son:

- **Servidor de desarrollo con *Hot Module Replacement* (HMR):** los cambios en el código se reflejan en el navegador sin recargar la página completa, preservando el estado.
- **Build optimizado para producción:** compila, minifica y divide el código en *chunks* para producir un paquete estático servible por cualquier servidor HTTP.
- **Configuración mínima y ecosistema de *plugins*.**

5.14. Tailwind CSS

Tailwind CSS [15] es un *framework* de estilos basado en *utility classes*: en lugar de escribir hojas de estilo (CSS) tradicionales, la apariencia se compone directamente en el marcado mediante clases atómicas (por ejemplo, *p-4*, *text-lg*, *bg-indigo-600*, *rounded-lg*).

Ventajas aprovechadas:

- Elimina la necesidad de nombrar clases semánticas cuando no aportan valor.
- El paso de *build* descarta las clases no utilizadas, produciendo hojas de estilo compactas.
- Aporta un sistema de diseño (espaciados, tipografía, paleta) coherente por defecto.
- Fomenta una interfaz responsiva mediante *breakpoint prefixes* (*sm:*, *md:*, *lg:*) y admite el prefijo *print:* para especializar la apariencia al momento de imprimir.

5.15. Bibliotecas complementarias del frontend

Alrededor de React se utilizan bibliotecas específicas para tareas concretas:

TanStack Query Gestor de estado de servidor. Se encarga de solicitar, cachear, revalidar e invalidar los datos que llegan del backend. Aporta reintento automático ante fallos transitorios, *polling* declarativo y sincronización entre pestañas.

React Hook Form Biblioteca de manejo de formularios que evita renderizados innecesarios y ofrece una API basada en *hooks*, ligera comparada con alternativas como *Formik*.

Zod Biblioteca de validación de esquemas con inferencia de tipos. Se combina con React Hook Form para validar en el cliente antes de enviar al servidor.

React Router Manejo del enrutamiento del lado del cliente. Permite construir SPAs con navegación sin recarga y *guards* de ruta según el rol.

axios Cliente HTTP con soporte de *interceptors*, utilizado para adjuntar automáticamente el *access token* JWT y refrescarlo cuando expira.

SheetJS (xlsx) Genera archivos *.xlsx* en el navegador para la descarga de reportes.

5.16. Patrones frontend: SPA con *guards*, *interceptores* y estado de servidor

El frontend se apoya en tres patrones que estructuran el flujo de datos y de navegación:

Route guard (guardián de ruta) Componente contenedor que, antes de renderizar la ruta que envuelve, verifica una condición (autenticación, rol) y decide entre renderizar los hijos o emitir una redirección declarativa (el elemento `Navigate` de *React Router*). El proyecto compone dos *guards* independientes — `ProtectedRoute` (chequea sesión) y `RoleRoute` (chequea rol) — para obtener `AdminRoute` y `ProfessorRoute`. La redirección declarativa se prefiere sobre `navigate()` imperativo dentro del *render* para evitar efectos secundarios en el ciclo de vida de React.

Interceptor (axios) Función que intercepta las peticiones o respuestas HTTP antes de que las reciba el código que las originó. En el proyecto, un *interceptor* de petición adjunta la cabecera `Authorization: Bearer` con el *access token*; un *interceptor* de respuesta detecta 401 `Unauthorized`, lanza un *refresh* del token y encola las peticiones concurrentes que caen en 401 para reintentarlas transparentemente con el nuevo *access token*.

Estado de servidor vs. estado local La distinción, popularizada por TanStack Query, separa los datos que *pertenecen* al servidor (y que el cliente cachea) del estado propio de la interfaz (formularios en edición, apertura de *modals*). El servidor es la fuente de verdad; el cliente sincroniza con estrategias como *stale-while-revalidate* (mostrar lo cacheado y refrescar en paralelo) y *polling* (revalidar cada N segundos, útil para vistas colaborativas).

5.17. Componentes de interfaz: *form-builder*, *cascada*, *acordeón*, *dashboard* y *badge*

El sistema utiliza patrones de interacción específicos, mencionados a lo largo del reporte:

Form-builder Constructor visual de formularios. Permite al administrador definir la estructura (secciones, preguntas de distintos tipos, opciones, puntuaciones) desde una interfaz gráfica en lugar de programar. Se contrapone a los formularios estáticos codificados en el frontend. En este proyecto es la vía por la que la Coordinación diseña las autoevaluaciones docentes.

Cascada (*drill-down*) Selección progresiva en la que cada elección restringe el conjunto de opciones del siguiente paso. En la vista pública, el visitante elige primero un periodo, luego un programa (licenciatura o posgrado) y por último una UEA, filtrando los documentos a mostrar.

Acordeón (*collapsible sections*) Componente que agrupa contenido en paneles expandibles y colapsables. Se usa para navegar por UEAs agrupadas por trimestre en la vista pública y para las listas de documentos del profesor agrupadas por periodo.

Dashboard Tablero con indicadores clave de desempeño (*KPIs*) y resúmenes agregados, pensado para lectura rápida. El sistema entrega dos dashboards: uno para el profesor (sus documentos por periodo activo) y uno para el administrador (métricas globales de cumplimiento).

Badge y chip Etiquetas visuales pequeñas que comunican el estado de un objeto (por ejemplo, "BORRADOR", "PUBLICADO", "ACTIVO"). En este proyecto los *chips* clicables del listado de periodos también actúan como controles para activar o desactivar cada uno de los tres *flags* de recurso.

5.18. Autenticación con JSON Web Tokens (JWT)

Un *JSON Web Token* (JWT) [16] es un formato compacto y firmado criptográficamente para transmitir información entre partes. Se define en el estándar *RFC 7519* y consta de tres partes codificadas en Base64URL y separadas por puntos: encabezado, *payload* (información del usuario y expiración) y firma (para verificar integridad).

En el proyecto la autenticación con JWT se implementa mediante *django-rest-framework-simplejwt* y sigue este flujo:

1. El usuario envía credenciales al endpoint `/api/v1/auth/login/`.
2. El servidor valida las credenciales y responde con dos tokens: uno de *acceso* (corta vida, típicamente 60 minutos) y uno de *refresco* (vida más larga, siete días).
3. El frontend envía el *access token* en el encabezado `Authorization: Bearer <token>` de cada petición posterior.
4. Cuando el *access token* expira, un *interceptor* del cliente HTTP solicita uno nuevo al endpoint `/api/v1/auth/refresh/` usando el *refresh token*.

5. Los *refresh tokens* rotan: cada renovación invalida el anterior [17].

Se prefirió JWT sobre el esquema tradicional de sesiones con *cookies* porque encaja mejor con APIs REST sin estado consumidas por un frontend desacoplado.

5.19. Control de acceso basado en roles (RBAC)

Role-Based Access Control (RBAC) es un modelo de autorización en el que los permisos no se asignan a usuarios individualmente sino a *roles*, y cada usuario recibe uno o varios roles. El proyecto define dos roles:

- **ADMIN:** gestiona catálogos, profesores, formularios de autoevaluación y consulta reportes globales.
- **PROFESOR:** gestiona sus propias cartas temáticas, requisitos y autoevaluaciones.

Los permisos se aplican en dos capas:

- En el **backend** mediante clases *DRF permission* (por ejemplo, `IsOwnerProfesorOrAdminReadOnly`) que verifican rol y propiedad del recurso.
- En el **frontend** mediante *guards* de ruta que redirigen si el rol no coincide, y renderizado condicional que oculta acciones no autorizadas.

La verificación en ambos lados es *defensa en profundidad*: el backend es la fuente de verdad; el frontend evita mostrar interfaces inútiles. El principio de defensa en profundidad, tomado de la ingeniería de seguridad, consiste en superponer varias capas independientes de protección para que la falla de una no comprometa al sistema.

5.20. Seguridad web y OWASP Top 10

El *OWASP Top 10* [18] es un documento consensuado por la *Open Web Application Security Project* que enumera las diez categorías de riesgo de seguridad más críticas en aplicaciones web. Sirve como *checklist* de referencia para desarrolladores. Las categorías atendidas explícitamente en este proyecto son:

- **A01 — Broken Access Control:** se mitiga con las verificaciones de rol y propiedad en el backend.
- **A02 — Cryptographic Failures:** contraseñas hasheadas con PBKDF2 (algoritmo por defecto de Django) y tokens firmados con HMAC-SHA256. El *hashing* es una función unidireccional que produce una huella de longitud fija a partir de la contraseña; PBKDF2 la refuerza aplicando miles de iteraciones y un *salt* aleatorio, encareciendo los ataques de diccionario.
- **A03 — Injection:** prevenida por el ORM (consultas parametrizadas) y por la validación de entrada en *serializers*.

- **A05 — *Security Misconfiguration***: en producción se establece `DEBUG=False` (la variable de Django que activa las trazas detalladas de error, útiles en desarrollo pero peligrosas en producción) y una *whitelist* explícita de *hosts* y de orígenes CORS.
- **A07 — *Identification and Authentication Failures***: se aplica *throttling* en el *login* (5 intentos por minuto, respuesta 429 `Too Many Requests` al superarse), rotación de *refresh tokens* y política de contraseñas fuerte.

5.21. Formatos de intercambio de datos

El sistema produce, consume o exporta datos en varios formatos, cada uno elegido por adecuarse a un caso de uso concreto:

JSON (*JavaScript Object Notation*) — formato textual ligero de pares clave–valor y arreglos anidados. Es el formato por defecto de la API REST (peticiones y respuestas) y de los *fixtures* de Django que precargan los catálogos institucionales.

JSONB — variante binaria e indexable de JSON que ofrece PostgreSQL; se usa dentro del modelo `Pregunta` para el campo `config` (por ejemplo, los límites de una escala lineal), evitando crear una tabla auxiliar por cada tipo de pregunta.

CSV (*Comma-Separated Values*) — formato tabular plano que separa campos por comas y filas por saltos de línea. Es el formato de intercambio con hojas de cálculo. El sistema lo usa para la importación masiva del catálogo de UEAs (`POST /uea/import-csv/` y el comando de gestión equivalente).

XLSX — formato binario de libros de Excel. Se genera del lado del cliente con *SheetJS*: el navegador arma un *workbook* con una hoja por periodo y lo descarga sin intervención del servidor. Esta elección reduce carga del backend y permite iterar rápidamente sobre el formato sin desplegar cambios.

YAML (*YAML Ain't Markup Language*) — formato textual jerárquico legible. En este proyecto sólo aparece como formato de salida del esquema *OpenAPI* en `/api/schema/`.

PDF (*Portable Document Format*) — se obtiene desde el propio navegador vía `window.print()` sobre las vistas públicas del documento, que aplican reglas CSS con el prefijo `print:` de Tailwind para producir una salida limpia.

5.22. Máquina de estados y ciclo de vida de artefactos

Una *máquina de estados finitos* modela un artefacto como un conjunto de estados discretos y un conjunto de transiciones válidas entre ellos, disparadas por acciones. Aplicar este patrón hace explícitas las reglas de negocio (qué acciones son válidas en qué momento) y evita estados inconsistentes.

El sistema usa dos máquinas de estados:

- Los **documentos académicos** (cartas y requisitos) tienen dos estados: `BORRADOR` (editable por el dueño) y `PUBLICADO` (visible al público, sólo despublicable por el dueño para volver a editar). La transición se dispara con la acción `cambiar-estado`.

- Los **formularios de autoevaluación** tienen tres estados: BORRADOR, PUBLICADO (aceptando respuestas) y CERRADO (histórico, sin recibir nuevas respuestas). Las acciones válidas son publicar, cerrar, reabrir, despublicar y publicar_revision. Esta última merece atención aparte por su efecto sobre el versionado (§ 5.23).

Modelar el ciclo como máquina de estados aporta idempotencia (aplicar publicar sobre un formulario ya publicado no cambia nada), inmutabilidad tras publicar (no se pueden alterar preguntas o pesos de un formulario publicado sin transitar por despublicar o publicar_revision) y trazabilidad de las transiciones.

5.23. Versionado inmutable y *snapshots* históricos

Dos técnicas complementarias garantizan que la información histórica sobreviva a los cambios administrativos posteriores:

Snapshot de identidad Al crear un documento, se copian el nombre y el correo del profesor en columnas dedicadas (profesor_nombre, profesor_correo). Si más adelante el usuario se elimina, la FK con on_delete=SET_NULL deja el documento sin dueño vivo, pero los *snapshots* preservan a quién pertenecía. Es una técnica de *denormalización controlada* que sacrifica una pequeña redundancia por trazabilidad institucional.

Versionado incremental de formulario Cada Formulario tiene un entero que indica la version del mismo (inicialmente 1). La acción publicar_revision incrementa esta versión al reabrir el formulario tras su cierre; las respuestas ya enviadas quedan atadas a la versión con la que se contestaron mediante el campo version_formulario de Respuesta. Además, cada RespuestaSeccion copia el peso de la sección al momento del envío, de modo que el desglose por sección permanece estable aunque el administrador ajuste pesos posteriormente. Este esquema hace posibles comparaciones históricas consistentes sin bloquear la evolución del instrumento.

5.24. Escalas de medición: Likert y niveles de desempeño

Una *escala de Likert* es una escala psicométrica en la que el respondiente indica su grado de acuerdo con una afirmación mediante una opción entre varias equiespaciadas (típicamente 1 a 5: “Muy en desacuerdo” ... “Muy de acuerdo” o “Nunca” ... “Siempre”). En el sistema esta escala aparece como el tipo de pregunta ESCALA_LINEAL y también, en modo compacto de matriz filas×columnas, como el tipo CUADRICULA.

Un *nivel de desempeño* es un rango porcentual con etiqueta descriptiva, observación y color (por ejemplo, “0–40 Insuficiente”, “40–70 Suficiente”, “70–90 Bueno”, “90–100 Sobresaliente”). Al enviar una autoevaluación, el sistema calcula el porcentaje global ponderado y busca el nivel cuyo rango lo contiene; adjunta el nivel y su observación al resultado. Este mecanismo permite convertir un puntaje continuo en una interpretación cualitativa comprensible para el profesor.

5.25. Metodología, hitos y control de versiones

El desarrollo del sistema se organizó por *hitos* incrementales (M0–M9), cada uno con un objetivo verificable y un *checkpoint* de revisión: M0 corresponde al andamiaje inicial del monorepo, y M9 al pulido final y la integración con la documentación. Dentro del módulo de autoevaluación — el más elaborado — se etiquetaron nueve incrementos “PR1” a “PR9” que corresponden a otros tantos *pull requests* en el historial de git, cada uno enfocado a una capacidad concreta (*form-builder* básico, tipos de pregunta, secciones ponderadas, niveles de desempeño, versionado, estadísticas, etc.).

El código se organiza como *monorepo*: un único repositorio git con dos subproyectos hermanos, *backend/* y *frontend/*. Los mensajes de *commit* siguen la convención *Conventional Commits* (prefijos *feat*, *fix*, *chore*, *docs*, *refactor*) para facilitar la lectura del historial y automatizar futuros *changelogs*.

La configuración por entorno se aloja en archivos *.env* (leídos con *python-decouple*) que definen credenciales de base de datos, la clave secreta de Django, el modo *DEBUG* y la lista blanca de CORS. El archivo *.env* nunca se versiona; en su lugar, un *.env.example* documenta las variables necesarias.

5.26. Pruebas automatizadas

Las pruebas automatizadas del proyecto se ejecutan con *pytest* y *pytest-django*. Los conceptos utilizados son:

- **Prueba unitaria:** verifica una función o unidad aislada.
- **Prueba de integración:** verifica la interacción de varios componentes (por ejemplo, una petición HTTP que atraviesa *view*, *serializer*, ORM y base de datos).
- **Prueba end-to-end (E2E):** verifica el comportamiento del sistema completo desde el punto de vista del usuario, usualmente atravesando frontend y backend. El script `scripts/smoke_test.py` ejerce los principales *endpoints* contra un servidor en ejecución como *smoke test* E2E.
- **Factory:** patrón para construir instancias de prueba consistentes; se emplea la biblioteca *factory-boy*.
- **Cobertura de código:** porcentaje del código ejecutado por las pruebas. Se mide con *coverage.py* a través del complemento *pytest-cov*.
- **Fixture (dos acepciones):** en el contexto de *pytest*, es una función que provee datos preparados a una prueba; en el contexto de Django, es un archivo (JSON o YAML) con datos precargables al inicializar el sistema. Ambas acepciones se usan en el proyecto y se distinguen en el texto por el contexto.

6. Desarrollo del proyecto

Esta sección narra el desarrollo del sistema *paso a paso*: de la arquitectura general al flujo operativo de cada módulo, cubriendo la construcción del modelo de datos, la gestión de profesores y catálogos, la habilitación de periodos por recurso, la elaboración y publicación de documentos, la construcción de formularios de autoevaluación con su sistema de puntuación, la descarga de reportes y la vista pública de exploración. La estructura del código se divide en dos monorepositorios lógicos: el `backend/` (Django) y el `frontend/` (React + Vite), ambos versionados con *git* bajo el directorio raíz `proyecto_terminal_cyad/`. Los fragmentos de código que ilustran cada mecanismo se recopilan en el Apéndice A y se citan por su *label* desde este capítulo.

6.1. Arquitectura general

El sistema sigue una arquitectura cliente–servidor de tres capas:

1. **Capa de presentación (frontend)**: aplicación SPA en React 19 servida por Vite 8; renderizada en el navegador; consume la *API REST* mediante *axios*; almacena el *JWT* en *localStorage* y gestiona el ciclo de vida de las sesiones (*refresh* automático transparente, expulsión al expirar el *refresh token*).
2. **Capa de aplicación (backend)**: servidor Django 5 que expone la *API REST* bajo el prefijo `/api/v1/`; orquesta la lógica de negocio en *ViewSet* de DRF; valida y serializa entradas con *Serializers*; documenta la API con *drf-spectacular* (`/api/docs/` y `/api/schema/`).
3. **Capa de persistencia**: base de datos PostgreSQL 16 accedida vía el ORM de Django, con migraciones versionadas por app y *constraints* declarativos que actúan como defensa en profundidad.

La Figura 1 muestra el diagrama de arquitectura y la Figura 2 el diagrama de casos de uso, con los actores *Administrador*, *Profesor* y *Público* y sus interacciones con los módulos funcionales.

6.1.1. Estructura de directorios

- `backend/`: proyecto Django. Contiene la configuración global (`config/`), las seis apps (`core`, `accounts`, `catalogos`, `documentos`, `autoevaluacion`, `reportes`), *scripts* operativos (`scripts/`), suite de pruebas (`tests/`), `conftest.py` con *fixtures* globales de *pytest* y `requirements.txt`.
- `frontend/`: aplicación React + Vite. Contiene `src/pages/` (por rol: `admin/`, `profesor/`, `publico/`), `src/components/ui/` (componentes reutilizables), `src/api/` (clientes HTTP), `src/auth/` (contexto de sesión y *guards*), `src/layouts/` y `src/utils/`.
- `docs/`: documentación complementaria; contiene este reporte (`docs/reporte/`) y los manuales (`docs/manuales/`).

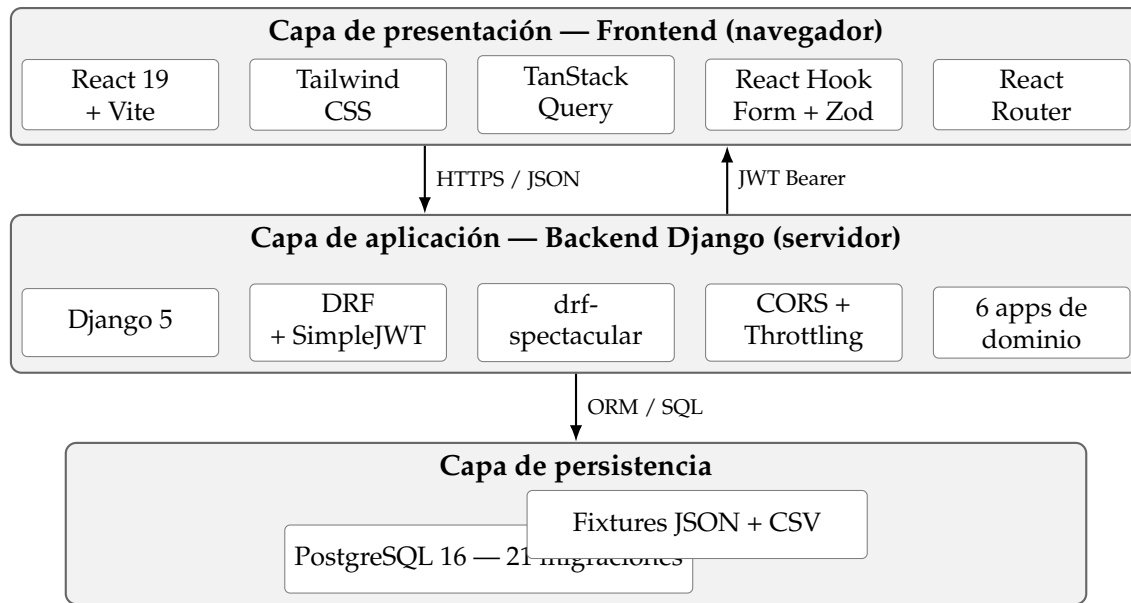


Figura 1. Arquitectura de tres capas del sistema.

6.2. Metodología y evolución del desarrollo

El desarrollo se organizó por *hitos* incrementales (M0–M9), cada uno con un objetivo verificable. M0 estableció el andamiaje del monorepo (Django + React inicializados, PostgreSQL conectado, JWT operativo). Los hitos M1 a M8 construyeron los módulos funcionales sucesivos y M9 corresponde al pulido, integración con la documentación y captura de evidencia visual.

Dos episodios reorientaron el trabajo y quedan reflejados en el historial de *git*:

1. **Rediseño de junio de 2026 (documentos aplanados):** la propuesta original modelaba la carta temática como un árbol de subtablas (Tema, Subtema, Bibliografía, CriterioEvaluacion). Tras validar con la Coordinación que la mayoría de los profesores redacta cada rubro como texto continuo, se decidió aplanar el modelo a doce `TextField` libres. La transición se materializó en la migración `documentos/0002_rediseño_documentos`, que redujo la complejidad del *form-builder* de cartas y aceleró el desarrollo del resto del módulo.
2. **Nueve incrementos “PR1–PR9” en autoevaluación:** el módulo más complejo se entregó en nueve *pull requests* sucesivos etiquetados en los mensajes de *commit*: PR1 (*form-builder* básico y tipos simples), PR2 (opciones puntuables), PR3 (tipo `ESCALA_LINEAL`), PR4 (tipo `CUADRICULA`), PR5 (secciones con peso y ponderación 100 %), PR6 (`NivelDesempeno`), PR7 (versionado y `publicar_revision`), PR8 (endpoints de estadísticas) y PR9 (duplicación cross-periodo y polución final).

La configuración por entorno se distribuye en un archivo `.env` local, leído por *python-decouple*; contiene `SECRET_KEY`, credenciales de PostgreSQL, `DEBUG` y `CORS_ALLOWED_ORIGINS`. El archivo nunca se versiona.

6.3. Modelo de datos

El modelo de datos se distribuye por app siguiendo la separación de dominios. Un diagrama entidad-relación completo se muestra en la Figura 3. En total, el sistema cuenta con 21 migraciones aplicadas distribuidas entre las apps (`accounts: 3`, `autoevaluacion: 5`, `catalogos: 7`, `documentos: 5`, y una inicial en `core` y `reportes`).

A continuación se describe cada entidad por app: campos que la conforman, restricciones declarativas y cardinalidad de sus relaciones. La notación empleada es la de *crow's foot* en formato “1-*N*” (uno a muchos), “1-1” (uno a uno), “0,1-*N*” (opcional a muchos) y “*N-M*” (muchos a muchos).

App core — infraestructura compartida

No define tablas físicas. Aporta dos *mixins* abstractos que la mayoría de los modelos concretos heredan:

- `TimeStampedModel`: agrega `created_at` (`auto_now_add`) y `updated_at` (`auto_now`).
- `EstadoActivoModel`: agrega el campo booleano `estado` para borrado lógico (*soft delete*).

Además provee la paginación por defecto (`DefaultPagination`), el *exception handler* unificado (`api_exception_handler`) y las clases de permisos DRF (`IsAdmin`, `IsProfesor`, `IsAdminOrReadOnly`, `IsOwnerProfesorOrAdminReadOnly`) usadas transversalmente.

App accounts — usuarios y perfiles de profesor

Fragmentos del código de estos modelos se recogen en el Apéndice A (“Backend — Perfil de Profesor y Periodo”).

Usuario Extiende `AbstractBaseUser + PermissionsMixin`. Campos: `email` (único, `USERNAME_FIELD`), `nombre`, `rol` ∈ {`ADMIN`, `PROFESOR`}, `is_active`, `is_staff`, `password` (hash PBKDF2 vía `set_password`), `created_at`, `updated_at`. Se acompaña de `UsuarioManager` con `create_user` y `create_superuser`.

Profesor Perfil extendido del usuario con `rol=PROFESOR`. Hereda de `TimeStampedModel` y `EstadoActivoModel`. Campos propios: `numero_economico` (único, opcional), `nombre_completo`, `correo_institucional` (único) y FK a `Departamento`. Un `save()` sobrescrito sincroniza `Usuario.is_active` con `Profesor.estado`, de modo que desactivar el perfil bloquea inmediatamente el inicio de sesión.

Relaciones:

- `Usuario` 1-1 `Profesor` (OneToOne con `on_delete=CASCADE`; borrar el `Usuario` borra el `Profesor`).
- `Departamento` 1-*N* `Profesor` (FK con `on_delete=SET_NULL`, opcional).

App **catalogos** — catálogos institucionales de CyAD

Departamento Departamento académico. Campos: `clave` (única), `nombre`, más los heredados de `TimeStampedModel` y `EstadoActivoModel`.

Licenciatura Programa de licenciatura. Campos: `clave` (única), `nombre`, `orden` (para ordenamiento en UI) y FK a `Departamento` (`SET_NULL`).

Posgrado Programa de posgrado (maestría, especialidad o doctorado). Mismos campos que `Licenciatura`.

Area Área curricular a la que pertenece una UEA (troncal, optativa de concentración, etc.). Campos: `nombre`, `descripcion`, con `unique_together=(nombre, descripcion)`.

UEA Unidad de Enseñanza-Aprendizaje. Campos: `clave` (normalizada a mayúsculas en `save()`), `nombre`, `trimestre` (entero 1–12 o cadena romana “VII–XII” para optativas), `tipo` $\in \{OBL, OPT, OTRO\}$, `creditos`, `liga` (URL oficial de la UEA). FKs con `on_delete=PROTECT`: a `Licenciatura`, `Posgrado` y `Area`, todas opcionales. Tres *constraints* declarativos: `CheckConstraint` `uea_xor_licenciatura_posgrado` (exactamente uno de los dos programas está poblado); y dos `UniqueConstraint` parciales sobre (`clave`, `licenciatura`) y (`clave`, `posgrado`).

Periodo Periodo trimestral (por ejemplo, “25-O”). Campos: `clave` (única), `fecha_inicio`, `fecha_fin`, tres booleanos independientes de habilitación por recurso (`activo_cartas`, `activo_requisitos`, `activo_autoevaluacion`), un booleano derivado `activo` (OR de los tres, sincronizado en `save()`) y `estado` (habilitación general). El `save()` sobrescrito garantiza la invariante “*a lo sumo un periodo activo por recurso*” apagando el *flag* correspondiente en los demás periodos dentro de la misma transacción.

Relaciones:

- Departamento 1–N `Licenciatura` y Departamento 1–N `Posgrado` (`SET_NULL`, opcional).
- `Licenciatura` 1–N UEA y `Posgrado` 1–N UEA (`PROTECT`, mutuamente excluyentes por el XOR).
- `Area` 1–N UEA (`PROTECT`, opcional).
- `Periodo` es referenciado desde `documentos` y `autoevaluacion` (véase abajo).

App **documentos** — cartas temáticas y requisitos de recuperación

Toda la app comparte la clase abstracta `DocumentoAcademicoBase` con los campos comunes: FK opcional a `Profesor` (`SET_NULL`, para preservar el documento como historial si el profesor es eliminado), *snapshots* de identidad `profesor_nombre` y

profesor_correo (rellenados automáticamente al crear el documento), FK a UEA y Periodo (ambas PROTECT), nombre_grupo, id_grupo, horario, modalidad $\in \{PRESENCIAL, REMOTO, MIXTO\}$, estado $\in \{BORRADOR, PUBLICADO\}$ y las marcas de tiempo de TimeStampedModel. Los métodos `puede_editar()` y `puede_eliminar()` exigen estado=BORRADOR.

CartaTematica Doce campos TextField libres para el contenido pedagógico: `descripcion_uea`, `objetivo_general`, `objetivos_particulares`, `contenido_sintetico`, `objetivos_aprendizaje`, `requerimientos`, `conocimientos_previos`, `modalidad_evaluacion`, `revisiones_a_sesorias`, `bibliografia`, `enlace_y_calendarizacion_actividades`. UniqueConstraint `unique_carta_profesor_periodo_uea_grupo` sobre (`profesor`, `periodo`, `uea`, `id_grupo`).

RequisitoRecuperacion Seis campos textuales: `lugar`, `duracion_aprox`, `fecha_hora`, `recursos_necesarios`, `requisitos` y `notas`. Misma UniqueConstraint compuesta que CartaTematica.

Relaciones (idénticas para ambos documentos):

- Profesor 0..1-N CartaTematica / RequisitoRecuperacion (SET_NULL).
- UEA 1-N CartaTematica / RequisitoRecuperacion (PROTECT).
- Periodo 1-N CartaTematica / RequisitoRecuperacion (PROTECT).

App autoevaluacion — *form-builder*, respuestas y scoring

Es la app con el modelo más rico. Se agrupa en tres bloques: definición del formulario, definición del baremo y captura de respuestas. Fragmentos representativos de estos modelos se listan en el Apéndice A (“Backend — Modelos del *form-builder*” y “Backend — Captura de respuestas y baremo”).

Definición del formulario.

Formulario Encabezado del instrumento. Campos: `titulo`, `descripcion`, FK a Periodo (PROTECT), estado $\in \{BORRADOR, PUBLICADO, CERRADO\}$, `version` (entero incrementable), `una_respuesta_por_profesor` (booleano), FK opcional a Usuario en `created_by` (SET_NULL), y timestamps de ciclo de vida `published_at` y `closed_at`. Los métodos `publicar`, `cerrar`, `reabrir`, `despublicar` y `publicar_revision` materializan la máquina de estados descrita en la Sección 6.8.2.

Seccion Bloque temático. Campos: FK a Formulario (CASCADE), `titulo`, `descripcion`, `orden`, `peso_decimal` (`max_digits=5`, `decimal_places=2`) cuya suma sobre todas las secciones debe ser 100 % ($\pm 0,01$) para poder publicar.

Pregunta Ítem individual. Campos: FK a Formulario (CASCADE) y FK opcional a Seccion (SET_NULL), tipo (ocho valores del TextChoices), texto, ayuda, obligatoria, orden y config (JSON dependiente del tipo: por ejemplo, {min, max, label_min, label_max, puntos_factor} para ESCALA_LINEAL o {puntos_si, puntos_no} para SI_NO). El conjunto TIPOS_NO_PUNTABLES = {TEXTO_CORTO, TEXTO_LARGO} marca las preguntas que se ignoran en el cálculo del puntaje.

OpcionPregunta Opción seleccionable. Campos: FK a Pregunta (CASCADE), texto, valor (identificador estable), puntos decimal y orden.

FilaCuadrícula Fila de una pregunta CUADRICULA. Campos: FK a Pregunta (CASCADE), texto y orden. Las columnas de la cuadrícula reutilizan OpcionPregunta.

Definición del baremo.

NivelDesempeno Rango porcentual con observación cualitativa. Campos: FK a Formulario (CASCADE), nombre, porcentaje_min y porcentaje_max decimales, observacion, color (clave para la UI: green|blue|yellow|red|gray) y orden.

Captura de respuestas.

Respuesta Envío por profesor. Campos: FK a Formulario (PROTECT), FK a Profesor (PROTECT), version_formulario (copia de Formulario.version al iniciar), estado ∈ {BORRADOR, ENVIADO}, enviado_at y los agregados calculados al enviar: puntaje_obtenido, puntaje_maximo, porcentaje y FK a NivelDesempeno (SET_NULL). UniqueConstraint sobre (formulario, profesor, version_formulario) que permite responder de nuevo tras una publicar_revision.

RespuestaPregunta Ítem de respuesta. Campos: FK a Respuesta (CASCADE), FK a Pregunta (PROTECT), valor_texto (para tipos abiertos) y opciones_seleccionadas (ManyToManyField a OpcionPregunta). unique_together=(respuesta, pregunta).

RespuestaCelda Elección columna-por-fila en una cuadrícula. Campos: FK a RespuestaPregunta (CASCADE), FK a FilaCuadrícula (PROTECT) y FK a OpcionPregunta (PROTECT). unique_together=(respuesta_pregunta, fila).

RespuestaSeccion Snapshot del puntaje por sección al enviar. Campos: FK a Respuesta (CASCADE), FK a Seccion (PROTECT), peso (copia del peso vigente al enviar), puntaje_obtenido, puntaje_maximo y porcentaje. El campo peso preservado permite reconstruir el desglose aunque el administrador re-ajuste los pesos y publique una revisión.

Relaciones (agrupadas por dueño):

- Periodo 1-*N* Formulario y Usuario 0,1-*N* Formulario (*creador*).
- Formulario 1-*N* {Seccion, Pregunta, NivelDesempeno, Respuesta}.
- Seccion 0,1-*N* Pregunta (una pregunta puede quedar huérfana si su sección se elimina, aunque la validación de publicación lo prohíbe).
- Pregunta 1-*N* OpcionPregunta y Pregunta 1-*N* FilaCuadricula.
- Profesor 1-*N* Respuesta y NivelDesempeno 0,1-*N* Respuesta.
- Respuesta 1-*N* RespuestaPregunta y Respuesta 1-*N* RespuestaSeccion.
- Pregunta 1-*N* RespuestaPregunta; Seccion 1-*N* RespuestaSeccion.
- RespuestaPregunta *N-M* OpcionPregunta (opciones_seleccionadas, con tabla intermedia autogenerada).
- RespuestaPregunta 1-*N* RespuestaCelda; FilaCuadricula 1-*N* RespuestaCelda; OpcionPregunta 1-*N* RespuestaCelda.

App reportes — vistas de agregación

No define modelos propios. Contribuye únicamente con *views* (véase el Módulo 6, Sección 6.9) que ejecutan agregaciones sobre las entidades de documentos, autoevaluacion y catalogos usando *aggregate*, *annotate*, *Count* y *Avg* del ORM.

6.4. Módulo 1: autenticación y gestión de usuarios

Este módulo, implementado en la app *accounts*, controla el acceso al sistema y la gestión del ciclo de vida de los usuarios.

6.4.1. Autenticación con JWT

El *endpoint* `POST /api/v1/auth/login/` extiende `TokenObtainPairView` de *simplejwt* con un *serializer* personalizado (Apéndice A, “Backend — Login con distinción de cuenta desactivada”) que:

- Diferencia *credenciales inválidas* de *cuenta desactivada* en los mensajes de error mediante un código `account_disabled`, para dar diagnóstico útil sin filtrar existencia de cuentas.
- Emite dos tokens: un *access token* con vigencia de 60 minutos y un *refresh token* con vigencia de 7 días. Los *refresh tokens* se rotan en cada uso (`ROTATE_REFRESH_TOKENS=True`).

- Aplica `throttle_scope="login"` para limitar a cinco intentos por minuto por dirección IP (véase § 6.12).
- Devuelve, junto con los tokens, un objeto `user` con `id`, `email`, `nombre`, `rol`, útil para poblar el contexto de sesión del frontend en la misma respuesta.

El *endpoint* `POST /api/v1/auth/refresh/` intercambia un *refresh token* válido por un nuevo par de tokens. El cliente frontend intercepta las respuestas HTTP 401 y dispara automáticamente el *refresh* antes de reintentar la petición original: mientras el *refresh* está en curso, las peticiones concurrentes que también fallan con 401 se encolan y se despachan de una sola vez con el nuevo *access token* (evita la “tormenta de *refresh*”). El *endpoint* `GET /api/v1/auth/me/` devuelve el perfil completo del usuario autenticado, incluyendo el *perfil_profesor* embebido cuando aplica.

6.4.2. Creación de un profesor con acceso al sistema

La creación de un profesor sigue un flujo operativo unificado desde el rol administrador:

1. El administrador entra a `/admin/profesores`, oprime *Nuevo profesor* y llena el formulario en la modal: *nombre completo*, *correo institucional* (que será el `USERNAME_FIELD`), *número económico*, *departamento* y *contraseña inicial*.
2. El frontend envía `POST /api/v1/profesores/crear-con-usuario/`. En una única transacción atómica el backend:
 - a) verifica si el correo ya corresponde a un usuario existente sin perfil de profesor asociado (“usuario huérfano”); si es así, lo reutiliza actualizando su rol y contraseña;
 - b) de lo contrario crea el `Usuario` con `rol=PROFESOR` y *hashea* la contraseña con `PBKDF2`;
 - c) crea el `Profesor` enlazado por FK al usuario y al departamento.
3. Devuelve `201 Created` con el profesor completo. Este puede iniciar sesión inmediatamente con el correo y la contraseña recibida.

La misma tabla ofrece dos acciones adicionales por fila:

- **Restablecer contraseña:** dispara una modal secundaria; `POST /api/v1/usuarios/{id}/set-password/` valida la nueva contraseña con la política estándar de Django (longitud mínima, no común, no numérica, no similar al perfil) y la guarda *hasheada*.
- **Activar / desactivar:** `POST /api/v1/usuarios/{id}/toggle-activo/` conmuta `is_active`. Al desactivar, el usuario queda con sesión expirada al vencer su *access token* actual (máximo 60 minutos) y no podrá refrescar ni iniciar sesión de nuevo.

Imagen sugerida: [IMAGEN PENDIENTE: captura de la modal “Crear profesor” desde /admin/profesores mostrando los campos *nombre*, *correo institucional*, *número económico*, *departamento* y *contraseña*. Ubicar en `Imagenes/screenshots/admin_crear_profesor.png` y referenciarla como `fig:admin_crear_profesor.`]

6.4.3. Roles y permisos

Los dos roles *ADMIN* y *PROFESOR* se materializan como valores de un `TextChoices` (mecanismo de Django que combina una enumeración con etiquetas legibles) en el modelo `Usuario`. La comprobación de rol se realiza:

- En **backend**, mediante *permission classes* de DRF: `IsAdmin` para operaciones exclusivas de administrador, `IsAdminOrReadOnly` para catálogos, `IsProfesor` para acciones del profesor y `IsOwnerProfesorOrAdminReadOnly` para restringir la escritura al dueño del recurso y permitir sólo lectura al administrador (Apéndice A, “Backend — Permisos por rol”). El chequeo tiene dos capas: `has_permission` (a nivel de `ViewSet`) y `has_object_permission` (a nivel de instancia).
- En **frontend**, mediante los *route guards* `AdminRoute` y `ProfesorRoute` (§ 5.16) que componen `ProtectedRoute + RoleRoute` y redirigen si el rol no coincide.

6.5. Módulo 2: catálogos institucionales

La app `catalogos` expone *ViewSets* de CRUD completo para `Departamento`, `Licenciatura`, `Posgrado`, `Area`, `UEA` y `Periodo`, todos regidos por `IsAdminOrReadOnly`: cualquier usuario autenticado los lee (necesario para poblar selectores en el frontend), pero sólo el administrador los modifica.

6.5.1. Precarga inicial y fixtures

Cinco *fixtures* JSON (ruta absoluta en el repositorio: `backend/catalogos/fixtures/*.json`) precargan los catálogos base: los cuatro departamentos de CyAD, las licenciaturas y posgrados activos, las áreas de conocimiento y cinco periodos históricos (24-I a 25-O). Se aplican con el comando `loaddata` de Django. Es necesario situarse primero en el directorio del proyecto Django (`backend/`, donde vive `manage.py`), activar el entorno virtual que aísla las dependencias (para que `python` y `manage.py` usen la versión de Django instalada por `pip install -r requirements.txt`, no el intérprete del sistema) y respetar el orden que dictan las claves foráneas (departamentos antes que licenciaturas y posgrados):

```
cd backend
.\.venv\Scripts\activate          # Windows (PowerShell / CMD)
# source .venv/bin/activate        # Linux / macOS
python manage.py loaddata \
    catalogos/fixtures/departamentos.json \
    catalogos/fixtures/licenciaturas.json \
    catalogos/fixtures/posgrados.json \
```



```
catalogos/fixtures/areas.json \
catalogos/fixtures/periodos.json
```

Las rutas indicadas son relativas a `backend/`. El comando `loaddata` es idempotente (usa la clave primaria de cada registro), por lo que se puede re-ejecutar sin duplicar datos: si un registro ya existe, lo actualiza. Esto reduce el tiempo de puesta en marcha del sistema a minutos, sin capturar manualmente los mismos datos. La creación del entorno virtual (`python -m venv .venv`), la instalación de dependencias y la configuración del archivo `.env` se documentan en el manual de instalación.

6.5.2. Importación masiva de UEAs desde CSV

Las UEAs son numerosas (una por asignatura por programa), por lo que se ofrecen dos vías de carga masiva idempotente:

- **Desde la interfaz:** `POST /api/v1/uea/import-csv/` con `multipart/form-data`. El backend lee las cabeceras `clave`, `nombre`, `programa_clave`, `trimestre`, `tipo`, `creditos`, `area_nombre`, `area_descripcion` y `url`, hace *upsert* usando cachés en memoria para evitar *queries* redundantes y detecta colisiones cuando la clave existe en licenciatura y posgrado simultáneamente.
- **Desde la línea de comandos:** el comando de gestión `python manage.py cargar_catalogos_csv` (Apéndice A, “Backend — Comando de importación CSV para UEA”) consume archivos CSV desde un directorio configurable y produce el mismo efecto. Es útil para automatizar la puesta en marcha en entornos nuevos.

6.5.3. Restricciones de integridad de UEA

El modelo UEA refuerza dos reglas del dominio como *constraints* declarativos (Apéndice A, “Backend — Constraint XOR y unicidad parcial de UEA”):

- `uea_xor_licenciatura_posgrado`: una UEA pertenece a una licenciatura o a un posgrado, nunca a ambos ni a ninguno (§ 5.11).
- Dos `UniqueConstraint` parciales que hacen la clave única por programa: la clave “11XY99” puede coexistir en una licenciatura y en un posgrado, pero no repetirse dentro del mismo programa.

Estas reglas se validan además en el *serializer* para producir mensajes de error legibles.

Imagen sugerida: [IMAGEN PENDIENTE: captura de `/admin/catalogos/uea` mostrando la tabla de UEAs, el buscador y el botón *Importar CSV*. Ubicar en `Imagenes/screenshots/admin_uea.png` y referenciarla como `fig:admin_uea`.]

6.6. Módulo 3: habilitación de periodos por recurso

Cada `Periodo` tiene tres *flags* de activación independientes:

activo_cartas Habilita la captura y publicación de cartas temáticas para ese periodo.

activo_requisitos Habilita la captura y publicación de requisitos de recuperación.

activo_autoevaluacion Habilita la creación de formularios de autoevaluación y la recepción de respuestas.

La separación en tres *flags* refleja el ritmo real de la Coordinación de Estudios: la ventana de captura de cartas se abre al inicio del trimestre, la de requisitos hacia la mitad, y la de autoevaluación al final. Los tres pueden estar activos en periodos distintos simultáneamente. Hacerlo de esta manera evita que los profesores se confundan y publiquen documentos en periodos que no corresponden a la fase de captura vigente.

6.6.1. Invariante “a lo sumo uno activo por recurso”

El sistema garantiza que sólo un periodo esté activo por recurso a la vez. Esto se implementa con un `save()` sobreescrito en `Periodo`: al persistir un periodo con `activo_cartas=True`, la misma transacción hace `activo_cartas=False` en todos los demás periodos. Lo mismo aplica a los otros dos *flags*. Un método de clase `Periodo.get_activo(recurso)` devuelve el periodo actualmente activo para un recurso dado, consumido por dashboards y por el asignador automático de periodo en los formularios de creación.

6.6.2. Interfaz de usuario

En `/admin/catalogos/periodos` cada fila del listado muestra tres *chips* (etiquetas) clicables (verde=activo, gris=inactivo). Al hacer clic sobre uno, el frontend dispara `PATCH /api/v1/periodos/{id}/` con el nuevo valor booleano; el resultado se refleja en toda la tabla porque los demás *chips* del mismo recurso pasan a gris automáticamente.

6.6.3. Efectos sobre otros módulos

La habilitación se propaga:

- El *serializer* de `CartaTematica` rechaza altas si el periodo destino no tiene `activo_cartas=True` (equivalente para requisitos y autoevaluación).
- El *ViewSet* de documentos calcula un *flag* derivado `puede_editar_ahora` que combina `estado=BORRADOR` con `periodo.activo_cartas=True`. El frontend lo usa para deshabilitar botones de edición cuando el periodo se cierra.
- El *endpoint* `GET /api/v1/periodos/activos/` devuelve un objeto `{cartas: {...}, requisitos: {...}, autoevaluacion: {...}}` consumido por los dashboards y por los formularios de creación de documento (para asignar automáticamente el periodo activo).

6.6.4. Eliminación protegida

Como `Periodo` es referenciado por múltiples entidades con FK `PROTECT`, eliminar un periodo requiere una operación cuidadosa:

1. El administrador consulta `GET /api/v1/periodos/{id}/preview-eliminacion/`, que devuelve el conteo de documentos y respuestas que se borrarían en cascada.
2. El `DELETE` está bloqueado si el periodo está activo para algún recurso.
3. Si el usuario confirma, `perform_destroy` realiza una cascada manual dentro de una transacción atómica en el orden *Respuesta* → *Formulario* → *CartaTematica* → *RequisitoRecuperacion* → *Periodo*.

Imagen sugerida: [IMAGEN PENDIENTE: captura de `/admin/catalogos/periodos` con tres *chips* por fila (Cartas, Requisitos, Autoevaluación) mostrando el estado activo/inactivo. Ubicar en `Imágenes/screenshots/admin_periodos.png` y referenciarla como `fig:admin_periodos`.]

6.7. Módulo 4: cartas temáticas y requisitos de recuperación

Este módulo, implementado en la app `documentos`, unifica el CRUD de los dos documentos porque comparten patrón: misma clase base abstracta, mismo ciclo de vida `BORRADOR` ↔ `PUBLICADO`, misma unicidad compuesta y mismas reglas de propiedad. Difieren en los campos específicos y en el *flag* de periodo que los habilita.

6.7.1. Creación y publicación desde el rol profesor

El flujo operativo del profesor es el siguiente:

1. Entra a `/profesor/cartas` (o `/profesor/requisitos`). Ve la lista de sus documentos agrupada por periodo con el componente `CollapsiblePeriodoCard` (§ 5.17, patrón acordeón). La tarjeta del periodo activo se abre automáticamente al terminar de cargar los periodos activos.
2. Oprime *Nueva carta*. El formulario (`CartaFormPage`) asigna automáticamente el periodo activo consultando `GET /periodos/activos/`; ese campo no es editable, evitando errores de captura.
3. Selecciona la UEA con un buscador libre que sugiere las diez mejores coincidencias por clave o nombre.
4. Llena *nombre de grupo*, *id de grupo*, *horario*, *modalidad* (presencial/remoto/mixto) y los doce campos de contenido de la carta (o los seis del requisito).
5. Elige *Guardar borrador* (`POST /cartas-tematicas/` o `PATCH /cartas-tematicas/{id}/`) o *Guardar y publicar*. Esta segunda opción encadena la actualización con `POST /cartas-tematicas/{id}/cambiar-estado/` para transicionar el documento a `PUBLICADO`.

Al crearse el documento, el *ViewSet* llena automáticamente los campos `profesor_nombre` y `profesor_correo` como *snapshots* de identidad (§ 5.23), extraídos del perfil del profesor autenticado. Estos *snapshots* sobreviven a la eventual desactivación o eliminación del usuario.

Las evidencias visuales del listado del menú principal del profesor y del formulario de creación de documentos se muestran en las Figuras 7, 8 y 9 de la Sección 7.

6.7.2. Reglas de propiedad y ediciones fuera de tiempo

El *ViewSet* de cartas usa el permiso `IsOwnerProfesorOrAdminReadOnly`: el profesor sólo ve y edita sus propios documentos; el administrador tiene acceso de sólo lectura a todos. Adicionalmente, un método privado `_profesor_puede_modificar(obj)` bloquea cualquier modificación — incluida la transición de estado — si el periodo del documento ya no tiene el *flag* correspondiente activo. Con esto, un profesor no puede alterar retroactivamente cartas de trimestres cerrados.

La *UniqueConstraint* compuesta (*profesor, periodo, uea, id_grupo*) impide, a nivel de motor de base de datos, que un profesor tenga dos documentos para el mismo grupo en el mismo trimestre. En el evento de una violación, DRF devuelve 400 *Bad Request* con el siguiente mensaje: “Ya existe un documento para este profesor, periodo, UEA y grupo”.

6.7.3. Vista del administrador

El administrador dispone de las páginas `/admin/documentos/cartas` y `/admin/documentos/requisitos` con la vista global de los documentos *publicados*, filtrable por periodo, con enlaces a la vista pública imprimible de cada uno. La interfaz es de sólo lectura: la publicación es responsabilidad del dueño; el administrador supervisa y descarga.

6.8. Módulo 5: *form-builder* y autoevaluación docente

Este es el módulo más elaborado del sistema. Se construyó en nueve incrementos PR1–PR9 (Sección 6.2) y culmina con el envío por parte del profesor de una respuesta puntuada y evaluada contra rangos de desempeño.

6.8.1. Flujo operativo del administrador (*form-builder*)

1. El administrador entra a `/admin/autoevaluacion` y ve la lista de formularios (con *polling* cada 15 s para reflejar cambios entre pestañas). Crea un formulario nuevo asociado a un *Periodo* que tenga `activo_autoevaluacion=True`.
2. Entra al constructor (`FormularioBuilderPage`). La interfaz se organiza en tres pestañas: *Preguntas*, *Niveles* y *Estadísticas*.
3. En *Preguntas* añade **secciones** con *peso decimal*. Un indicador visual muestra en tiempo real “ Σ pesos = X %” con marca de verificación cuando la suma es exactamente 100 (tolerancia $\pm 0,01$) o advertencia en caso contrario.

4. Añade preguntas dentro de cada sección. El sistema soporta ocho tipos:

- a) Texto corto (una línea).
- b) Texto largo (párrafo).
- c) Opción única (*radio*).
- d) Casillas (*checkboxes* múltiples).
- e) Sí/No.
- f) Escala lineal (Likert 1– N con etiquetas configurables; § 5.24).
- g) Lista desplegable.
- h) Cuadrícula (matriz filas $\times$ columnas para preguntas Likert compactas).

Para los tipos con opciones, define *puntos* por opción; para `ESCALA_LINEAL` configura `min`, `max`, `puntos_factor`; para `CUADRICULA` define filas y opciones (columnas).

5. En *Niveles* define los *niveles de desempeño*: rangos porcentuales con etiqueta, observación textual y color (por ejemplo, “0–40 Insuficiente”, “40–70 Suficiente”, “70–90 Bueno”, “90–100 Sobresaliente”).
6. Cuando la estructura está completa, oprime *Publicar*. El backend valida la estructura (véase la transición `publicar` en la Sección 6.8.2) y transiciona el formulario a `PUBLICADO`.

6.8.2. Máquina de estados del formulario

Un Formulario vive en una máquina de estados con tres estados (`BORRADOR`, `PUBLICADO` y `CERRADO`) y cinco transiciones etiquetadas, además de una acción transversal `duplicar`. La Figura 4 resume el diagrama; la implementación de las transiciones se encuentra en el Apéndice A (“Backend — Ciclo de vida de Formulario”).

Transiciones que preservan la versión.

- `BORRADOR` $\rightarrow$ `PUBLICADO`: acción `publicar`. Valida que haya al menos una sección, que la suma de pesos sea $100 \pm 0,01$ %, que no existan preguntas huérfanas (sin sección) y que cada sección con peso > 0 tenga al menos una pregunta puntuable. Registra `published_at`.
- `PUBLICADO` $\rightarrow$ `CERRADO`: acción `cerrar`. Deja de recibir respuestas. Registra `closed_at`.
- `CERRADO` $\rightarrow$ `PUBLICADO`: acción `reabrir`. Se emplea cuando el administrador cerró el formulario anticipadamente o por error, y desea volver a aceptar respuestas del **mismo instrumento** (mismos ítems, mismos pesos, misma *version*). Las respuestas ya recibidas se conservan y siguen contabilizando; los profesores que no habían respondido pueden hacerlo sin fricción.

- PUBLICADO/CERRADO → BORRADOR: acción `despublicar`. Permite editar preguntas y pesos; no incrementa la versión ni borra respuestas.

Transición con nueva versión.

- CERRADO → PUBLICADO (versión $n + 1$): acción `publicar_revision`. Se utiliza cuando, tras haber cerrado el formulario, el administrador **modificó la estructura** (agregó/quitó preguntas, ajustó pesos, cambió opciones u observaciones de niveles) y publica el instrumento corregido. A diferencia de `reabrir`, **incrementa versión**: las respuestas anteriores quedan asociadas a su versión histórica y no cuentan para el conteo de cumplimiento de la nueva; cada profesor debe volver a responder la revisión.

Acción transversal.

- `duplicar`: clona la estructura completa (secciones, preguntas, opciones, filas y niveles) a otro periodo, dentro de una transacción atómica, sin arrastrar respuestas. Puede ejecutarse desde cualquier estado y facilita reutilizar un formulario ya afinado en trimestres sucesivos.

6.8.3. Sistema de puntuación (*scoring ponderado*)

Cuando el profesor envía una respuesta, la acción `POST /respuestas/{id}/enviar/` desencadena el cálculo del puntaje. El algoritmo, resumido en el Apéndice A (“Backend — Scoring ponderado de autoevaluación”), procede así:

1. Para cada `Seccion`, recorre sus `Preguntas` puntables e ignora las de tipo `TEXTOCORTO/TEXTOLARGO` (colección `TIPOS_NO_PUNTABLES`).
2. Suma los puntos obtenidos y el máximo posible según el tipo:
 - `CUADRICULA`: $\text{máximo} = (\text{mayor } \textit{puntos} \text{ entre las opciones}) \times (\text{número de filas})$; $\text{obtenido} = \text{suma de puntos de cada celda respondida}$.
 - `ESCALA_LINEAL`: $\text{máximo} = \text{max} \times \text{puntos_factor}$; $\text{obtenido} = \text{valor respondido} \times \text{puntos_factor}$.
 - `SI_NO`: puntos configurados para la opción elegida (`puntos_si` o `puntos_no`).
 - `CASILLAS`: suma de puntos de todas las opciones marcadas.
 - `OPCION_UNICA / LISTA_DESPLEGABLE`: puntos de la opción seleccionada; $\text{máximo} = \text{mayor } \textit{puntos} \text{ entre las opciones}$.
3. Calcula el porcentaje por sección: $\text{porcentaje}_s = (\text{obtenido}_s / \text{máximo}_s) \times 100$.
4. Calcula el porcentaje ponderado global sumando, sobre todas las secciones, $\text{porcentaje}_s \times (\text{peso}_s / 100)$.

5. Persiste un `RespuestaSeccion` por sección con `puntaje`, `puntaje_maximo`, `porcentaje` y una **copia del peso** de la sección. Este *snapshot* garantiza que el desglose histórico permanezca estable aun si el administrador ajusta los pesos y publica una revisión.
6. Determina el `NivelDesempeno` correspondiente buscando el rango [`porcentaje_min`, `porcentaje_max`] que contiene al porcentaje global; adjunta el nivel y su observación al resultado.

6.8.4. Privacidad de la ponderación mientras se contesta

El sistema separa dos *serializers* de `OpcionPregunta`: `OpcionPreguntaSerializer` (para el administrador) expone el campo `puntos`; `OpcionPreguntaPublicSerializer` (para el profesor mientras contesta) lo omite. Así el respondiente no puede optimizar su elección conociendo la ponderación de antemano. Al enviar, el profesor recibe el resultado completo con desglose.

6.8.5. Flujo operativo del profesor

1. El profesor entra a `/profesor/autoevaluacion`. Ve la lista de formularios disponibles con un *badge* por estado (Pendiente, En progreso, Respondido, Periodo cerrado) y *polling* cada 15 s.
2. Abre un formulario. El `QuestionRenderer` (Apéndice A, “Frontend — Renderizador dinámico de preguntas”) despacha por `pregunta.tipo` y produce el control apropiado (*radio*, *checkbox*, *select*, escala 1–N, cuadrícula o *textarea*).
3. Puede *Guardar borrador* tantas veces como quiera (POST `/respuestas/` inicialmente, luego PATCH `/respuestas/{id}/`).
4. Cuando termina, oprime *Enviar* (POST `/respuestas/{id}/enviar/`). Si faltan respuestas obligatorias, el backend responde con `preguntas_faltantes` y el frontend resalta cada pregunta faltante en rojo y hace *scroll* suave a la primera.
5. Al éxito, se muestra un *ResultCard* con el puntaje obtenido, el porcentaje global, el nivel de desempeño asignado, la observación textual del nivel y el **desglose por sección** (porcentaje de la sección y aporte al total).

Imágenes sugeridas: además de la Figura 13 y la Figura 10 de la Sección 7, [IMAGEN PENDIENTE: captura del panel de *Niveles de desempeño* con los rangos y sus colores. Ubicar en `Imagenes/screenshots/admin_niveles_desempeno.png` como `fig:admin_niveles_desempeno`] y [IMAGEN PENDIENTE: captura del *ResultCard* tras enviar la autoevaluación, mostrando puntaje, porcentaje, nivel y desglose por sección. Ubicar en `Imagenes/screenshots/profesor_resultado_autoeval.png` como `fig:profesor_resultado_autoeval`].

6.9. Módulo 6: reportes de cumplimiento y descargas XLSX

6.9.1. Endpoints de agregación

La app `reportes` expone cinco endpoints de sólo lectura, todos restringidos al rol *Administrador*:

GET /api/v1/reportes/dashboard/?periodo={id} Métricas globales: total de profesores activos, conteo de cartas y requisitos por estado (BORRADOR/PUBLICADO), y desglose de formularios y respuestas de autoevaluación.

GET /api/v1/reportes/cumplimiento/?periodo=&departamento= Detalle de cumplimiento por departamento: para cada uno, cuenta profesores activos y cuántos publicaron carta y requisito en el periodo, con porcentajes.

GET /api/v1/reportes/cumplimiento-licenciatura/ Idem, agrupado por licenciatura, con conteos de UEA cubiertas.

GET /api/v1/reportes/autoevaluacion-profesores/ Lista de profesores con su porcentaje del último formulario, indicando quién ya envió y quién no.

GET /api/v1/reportes/autoevaluacion/ Estadísticas descriptivas por formulario: respuestas enviadas, total de profesores, tasa de respuesta, promedio general de porcentaje, distribución por nivel de desempeño y promedio por sección — este último calculado con `RespuestaSeccion.objects.filter(...).aggregate(Avg("porcentaje"))`.

Todos los agregados se calculan con la API de agregación del ORM (`aggregate`, `annotate`, `Count`, `Avg`), sin ejecutar SQL manual, aprovechando la parametrización automática frente a inyección.

6.9.2. Generación de XLSX del lado del cliente

Para las descargas tabulares el frontend usa *SheetJS* (biblioteca `xlsx`), evitando cargar al backend con generación de binarios (Apéndice A, “Frontend — Generación de XLSX por periodo”). El patrón adoptado es **una hoja por periodo**:

1. El administrador entra a `/admin/reportes` y elige tipo de reporte (cartas, requisitos o autoevaluación), periodo (o *Todos*) y opcionalmente departamento.
2. El frontend pide los datos ya publicados con `page_size=500` y filtros aplicados: `GET /api/v1/publico/cartas/?estado=PUBLICADO&periodo={id}&profesor__departamento={id}` (o su equivalente).
3. Agrupa las filas en un `Map<periodo_clave, rows[]>`.
4. Crea el libro con `XLSX.utils.book_new()` y añade una hoja por grupo con `XLSX.utils.json_to_sheet(rows.map(rowToExcel))` y `XLSX.utils.book_append_sheet(wb, ws, sheetName(clave))`.

5. Dispara la descarga en el navegador con `XLSX.writeFile(wb, filename)`.

El compromiso es que el navegador procesa la conversión — aceptable para volúmenes de hasta ~500 filas por reporte. La ventaja es que se puede iterar sobre el formato de las hojas sin desplegar cambios en el servidor.

La evidencia visual del panel de reportes está en la Figura 14.

6.10. Módulo 7: vista pública de exploración

Este módulo permite a cualquier persona consultar cartas y requisitos publicados sin autenticarse. Se apoya en un conjunto de *endpoints* bajo el prefijo `/api/v1/publico/` con permiso `AllowAny` y `authentication_classes=[]`, sin paginación y filtrados a documentos en estado `PUBLICADO`.

6.10.1. Cascada de tres pasos

La página `ExplorarPage (/publico/explorar/:tipo con tipo ∈ {cartas, requisitos})` implementa una cascada (§ 5.17):

1. **Periodo:** se listan los periodos que tengan documentos publicados del tipo elegido mediante `GET /publico/periodos/?tipo=carta`. Esto evita mostrar periodos vacíos.
2. **Programa:** el visitante elige entre licenciaturas y posgrados vía `GET /publico/licenciaturas/` y `GET /publico/posgrados/`.
3. **UEA:** se cargan las UEA con documentos disponibles vía `GET /publico/uea/?licenciatura={id}&periodo={id}&con_documentos_de=carta`. Para licenciaturas, la interfaz las agrupa por *trimestre* (patrón acordeón) o por nombre del área optativa; para posgrados, se muestran en un bloque único.

Los documentos publicados por cada UEA se cargan bajo demanda (*lazy load*) al expandir la sección correspondiente, usando la propiedad `enabled: expanded` de *TanStack Query*. Esto evita descargar cientos de documentos en la carga inicial.

6.10.2. Vista imprimible

Al hacer clic sobre un documento, la aplicación abre `/publico/cartas/{id}` o `/publico/requisitos/{id}`. Estas páginas usan `PublicDocumentLayout`, un envoltorio que aplica clases `Tailwind` con el prefijo `print:` (por ejemplo, `print:hidden` en la barra de acciones, `print:bg-white` en el fondo) para producir un PDF limpio desde el diálogo de impresión del navegador (`window.print()`). No se usan bibliotecas de generación de PDF del lado del servidor.

Un cliente `axios` separado (`publicClient` en `src/api/publico.js`) **sin** interceptores de JWT sirve estos endpoints, para evitar enviar `Authorization` cuando el usuario está autenticado en otra pestaña. El backend igualmente ignoraría el token porque los endpoints públicos usan `AllowAny`, pero el aislamiento simplifica el razonamiento.

La evidencia visual está en la Figura 15 de la Sección 7. La vista de detalle imprimible (`fig:publico_carta`) está pendiente de captura: [IMAGEN PENDIENTE: captura de `/publico/cartas/{id}` con *banner* del documento, campos de contenido y botón *Imprimir*. Ubicar en `Imagenes/screenshots/publico_carta.png`].

6.11. Aspectos transversales

6.11.1. Documentación viva de la API

El complemento *drf-spectacular* analiza las anotaciones de los *ViewSets* y *serializers* y genera un documento OpenAPI (§ 5.9) expuesto en `/api/schema/` (YAML descargable) y `/api/docs/` (Swagger UI interactivo). Esto elimina la clase de errores que produce mantener documentación manual desincronizada.

6.11.2. Formato de error unificado

Un *exception handler* personalizado (`core.exceptions.api_exception_handler`) envuelve todas las respuestas de error de DRF como `{success: false, status_code: N, errors: {...}}`. El frontend consume este contrato con una utilidad `parseApiError()` que prioriza campos conocidos (`preguntas_faltantes`, `detail`, `non_field_errors`) para producir mensajes legibles.

6.11.3. Pruebas automatizadas

El backend cuenta con una suite exhaustiva de pruebas construida con *pytest* 8, *pytest-django* 4 y *factory-boy* 3. La distribución aproximada de líneas de test por app es:

- `tests/test_auth.py`: 491 líneas.
- `tests/test_autoevaluacion.py`: 2 044 líneas.
- `tests/test_catalogos.py`: 563 líneas.
- `tests/test_documentos.py`: 698 líneas.
- `tests/test_reportes.py`: 390 líneas.
- **Total**: aproximadamente 4 186 líneas.

El archivo `backend/conftest.py` incluye una *fixture autouse* que limpia el *cache* de Django antes de cada prueba, evitando que el *throttling* acumulado del *login* contamine tests posteriores. La ejecución típica de la suite completa toma unos minutos y no requiere infraestructura adicional (usa una base de datos de test aislada). Los detalles del reporte de cobertura se presentan en la Sección 7.

6.12. Seguridad

Las medidas de seguridad implementadas siguen las recomendaciones de *OWASP Top 10* [18]:

- **Modo DEBUG seguro por defecto:** `DEBUG=False` salvo que `.env` lo active explícitamente (`config/settings.py`), evitando la exposición accidental de trazas y variables en producción.
- **Rate limiting del login:** cinco intentos por minuto vía `ScopedRateThrottle` de DRF con `throttle_scope="login"`. Después del quinto intento fallido, el endpoint responde 429 Too Many Requests.
- **Validación de contraseñas:** se aplican los cuatro validadores estándar de Django (similitud con datos de usuario, longitud mínima, contraseñas comunes, contraseñas numéricas).
- **Hashing de contraseñas:** PBKDF2 con SHA-256 por defecto (algoritmo estándar de Django para *hashing* de contraseñas), reforzado con *salt* aleatorio.
- **CORS whitelist:** los orígenes permitidos se configuran vía `CORS_ALLOWED_ORIGINS` y se envían credenciales sólo con los que estén en la lista.
- **Autenticación por JWT rotativo:** el *refresh* rota en cada uso, minimizando el impacto de una filtración; los tokens se firman con HMAC-SHA256.
- **Diferenciación de rol y permisos por ViewSet:** no hay endpoints sin permiso explícito; el *default* global es `IsAuthenticated`. La escritura sobre recursos propios se restringe adicionalmente con `IsOwnerProfessorOrAdminReadOnly`.
- **Endpoints públicos aislados:** bajo el prefijo `/api/v1/publico/`, expuestos con permiso `AllowAny` pero limitados a lectura de documentos en estado *PUBLICADO*, sin exponer campos internos ni contadores.
- **Prevención de inyección SQL:** garantizada por el uso exclusivo del ORM de Django; no se ejecuta SQL crudo con interpolación de cadenas en ninguna vista.

7. Resultados

Esta sección presenta los resultados obtenidos con evidencia cuantitativa (métricas del código, tests, cobertura y endpoints) y evidencia cualitativa (capturas de pantalla del sistema en operación con datos de *seed*).

7.1. Métricas del código y de la base de datos

La Tabla 2 resume el volumen del sistema entregado.

Tabla 2. Métricas del código entregado.

Componente	Cantidad	Notas
Apps Django	6	core, accounts, catalogos, documentos, autoevaluacion, reportes
Migraciones aplicadas	21	Distribuidas entre las apps de dominio
Endpoints REST de la API v1	~45	Documentados automáticamente vía OpenAPI en /api/docs/
Modelos concretos	14	Sin contar clases abstractas TimeStampedModel y DocumentoAcademicoBase
Dependencias Python	11	Ver backend/requirements.txt
Dependencias JavaScript (prod)	11	Ver frontend/package.json
Tipos de pregunta soportados	8	TEXTO_CORTO, TEXTO_LARGO, OPCION_UNICA, CASILLAS, SI_NO, ESCALA_LINEAL, LISTA_DESPLEGABLE, CUADRICULA
Componentes UI reutilizables	10+	Button, Alert, Badge, Table, Modal, FormField, Loading, EmptyState, CollapsiblePeriodoCard, entre otros
Fixtures precargados	5	Departamentos, licenciaturas, posgrados, áreas, periodos (catalogos/fixtures/)

7.2. Métricas de pruebas

La Tabla 3 muestra el volumen de la suite de pruebas del backend.

Tabla 3. Volumen de pruebas por módulo.

Archivo de pruebas	Líneas	Cobertura módulo
tests/test_auth.py	491	Autenticación y usuarios
tests/test_autoevaluacion.py	2 044	Autoevaluación (form-builder + scoring)
tests/test_catalogos.py	563	Catálogos institucionales
tests/test_documentos.py	698	Cartas y requisitos
tests/test_reportes.py	390	Reportes de cumplimiento
Total backend	≈4 186	

La ejecución completa de la *suite* arroja **196 pruebas exitosas de 196** (100 %) y una **cobertura de código del 94 %** medida con `coverage.py` (comando `pytest -cov=. -cov-report=html`). El reporte HTML detallado se incluye en la carpeta de entregables bajo `cobertura/`.

7.3. Evidencia visual del sistema en operación

Las siguientes capturas se tomaron con el sistema ejecutándose localmente contra la base de datos de *seed* (`python scripts/seed_demo.py`). Las credenciales usadas son las del entorno de desarrollo (`admin@cyad.uam.mx` / `Admin1234!` para el administrador; un profesor de prueba también sembrado).

7.3.1. Autenticación

7.3.2. Vista Docente

7.3.3. Vista Administrador

7.3.4. Vista pública

7.4. Documentación de la API

La API queda documentada automáticamente vía *drf-spectacular*. En `/api/docs/` se accede a la interfaz *Swagger UI* interactiva, y en `/api/schema/` al esquema OpenAPI en YAML descargable.

8. Análisis y discusión de resultados

Esta sección compara lo entregado con lo comprometido en la propuesta, discute las decisiones de diseño que se apartaron de aquélla y examina las limitaciones del sistema actual.

8.1. Comparación entregado vs. propuesta

La Tabla 4 sintetiza el grado de cumplimiento de cada componente del sistema respecto de lo que se prometió en la propuesta.

Tabla 4. Grado de cumplimiento por componente respecto a la propuesta.

Componente	Cumplimiento	Observaciones
Módulo de autenticación	Completo con matices	Identidad por email en lugar de número económico; <i>rate limiting</i> sí; <i>reset</i> autoservicio no.
Cartas temáticas (CRUD)	Completo	Ciclo Borrador/Publicado, unicidad, permisos, snapshots de identidad.
Cartas temáticas (contenido estructurado)	Reemplazado	Los submodelos Tema/Subtema/Bibliografía/CriterioEvaluacion se reemplazaron por texto libre tras el rediseño de junio 2026.
Requisitos de recuperación (CRUD)	Completo	Mismo patrón que cartas.
Requisitos de recuperación (RequisitoItem)	Reemplazado	También aplanado en el rediseño.
Exportación a PDF/-DOCX	No entregado	Se implementó exportación a XLSX en el cliente para reportes; documentos usan <code>window.print()</code> del navegador.
Autoevaluación docente (formulario)	Ampliado	Se entregó un <i>form-builder</i> configurable con ocho tipos de pregunta y ocho iteraciones (PR1-PR9).
Autoevaluación (scoring y observaciones)	Completo	<i>Scoring</i> ponderado por sección con niveles de desempeño automáticos.
Autoevaluación (PDF descargable por el docente)	No entregado	El resultado se muestra en pantalla; no hay descarga en PDF.
Autoevaluación (privacidad admin)	Ajustado	El admin ve porcentajes agregados y quién respondió; el detalle cualitativo queda para el propio profesor.

Componente	Cumplimiento	Observaciones
Reportes de cumplimiento	Completo	Dashboards por departamento y por licenciatura; export XLSX.
Reportes en PDF	No entregado	Ver observación de exportación.
Base de datos PostgreSQL	Completo	21 migraciones, <i>constraints</i> declarativos, fixtures y CSV para catálogos.
Autenticación JWT + validación de contraseñas + CORS whitelist	Completo	Ver Sección 6.12.
Manual de usuario y de instalación	Entregados	En docs/manuales/ (documentos separados).
Vista pública de exploración	Extra (no comprometido)	Cascada periodo→programa→UEA.
Catálogos administrables completos	Extra (no comprometido)	Departamentos, licenciaturas, posgrados, áreas, UEA (con CSV) y periodos.

8.2. Discusión de las brechas frente a la propuesta

8.2.1. Identidad por email en lugar de número económico

La propuesta indicaba autenticación con número económico + contraseña. En el sistema entregado, el `USERNAME_FIELD` es `email` y el número económico existe como campo opcional del perfil `Profesor`. La decisión se sostuvo por tres razones: (a) el email institucional es único, garantizado y conocido por todo el personal; (b) el número económico no siempre está disponible en el momento del alta y varía menos entre trámites; y (c) el email facilita la comunicación (envío de correos institucionales, futuras notificaciones, reset de contraseña). El impacto para el usuario es mínimo: en la práctica ingresa su email institucional en lugar de su número.

8.2.2. Ausencia de exportación a PDF/DOCX en backend

Ésta es la brecha de mayor visibilidad. La propuesta prometía generación de PDFs y DOCXs para cartas, requisitos, resultados de autoevaluación y reportes. En el sistema entregado, la única forma de *generar un PDF* de una carta o requisito es abrir la vista pública e imprimir a PDF desde el navegador (`window.print()`); la única exportación estructurada es la descarga a XLSX de los reportes, generada en el cliente con *SheetJS*. Las razones prácticas fueron: (a) evitar la dependencia de librerías pesadas (*WeasyPrint* requiere GTK; *ReportLab* requiere maquetación manual); (b) priorizar el flujo de operación diaria (captura, publicación, reportes en línea) sobre entregables descargables; y (c) concentrar el esfuerzo en el módulo de autoevaluación, técnicamente más complejo. En la Sección 9 se marca la generación nativa de PDFs como trabajo futuro prioritario.

8.2.3. Cartas y requisitos aplanados

Los submodelos Tema, Subtema, Bibliografía y CriterioEvaluacion propuestos originalmente para estructurar el contenido de la carta temática se eliminaron mediante la migración `documentos/0002_rediseño_documentos` y se sustituyeron por campos `TextField` libres. La motivación fue la ausencia de una plantilla oficial de CyAD que estableciera con carácter normativo qué campos debían existir y con qué cardinalidad. Con contenido libre, cada profesor organiza su carta como considere conveniente, y la Coordinación puede definir posteriormente un formato estándar sin necesidad de migrar datos.

8.2.4. Sin reset de contraseña autoservicio

Sólo el administrador puede restablecer la contraseña de un profesor. Un flujo de *forgot password* con token por correo se documenta como línea de trabajo futuro pero no se entregó porque el envío de correos requiere infraestructura adicional (SMTP institucional configurado) que no estuvo disponible en el trimestre de desarrollo.

8.3. Elementos entregados que no estaban en la propuesta

Estos componentes se incorporaron a solicitud durante el desarrollo y elevan el valor del sistema:

- **Vista pública *Explorar*:** cascada periodo→programa→UEA para consulta de cartas y requisitos publicados. Beneficia a estudiantes y personal externo.
- **Form-builder de autoevaluación:** en lugar de un formulario fijo, el administrador diseña formularios con ocho tipos de pregunta, secciones ponderadas y versionado.
- **Catálogos administrables completos** con importación CSV.
- **Snapshots de identidad del profesor** en los documentos, para preservar autoría aunque el registro sea desactivado.
- **Suite de pruebas de ~4 200 líneas** en el backend.

8.4. Limitaciones del sistema actual

- **Sin pruebas en el frontend:** no hay *Vitest*, *Jest* ni *React Testing Library* configurados. Los flujos críticos se validan sólo por inspección manual y por los tests de la API que consumen.
- **Sin CI/CD:** no hay pipeline configurado; las pruebas y *lints* se ejecutan manualmente antes de cada *commit*.
- **Sin backups automáticos configurados:** la política de respaldo dependerá de la infraestructura de despliegue.

- **Sin métricas de uso:** no se registra actividad de usuarios para análisis posterior (no hay dashboard de *engagement* ni logs estructurados).
- **Sin accesibilidad medida:** aunque *Tailwind* y los componentes reutilizables favorecen la accesibilidad, no se realizó una auditoría formal con *Lighthouse* o *axe-core*.

Estas limitaciones no impiden la operación del sistema pero delimitan expectativas y sugieren prioridades para iteraciones futuras.

9. Conclusiones

El proyecto cumplió con el objetivo general planteado en la propuesta: se desarrolló e implementó un sistema web funcional que permite a los profesores de la División de Ciencias y Artes para el Diseño (CyAD) de la UAM-A gestionar cartas temáticas, requisitos de recuperación y responder autoevaluaciones, con acceso diferenciado por rol y reportes de cumplimiento para el personal administrativo. Los cuatro objetivos específicos se cumplieron sustancialmente, con matices que se detallan en la Sección 8 y que sintetizamos aquí.

9.1. Cumplimiento por objetivo

1. **Autenticación con roles:** cumplido. El sistema autentica mediante *JWT*, diferencia dos roles y aplica *throttling* sobre el *login*. El único matiz es que la identidad primaria del usuario es el *email* y no el número económico, decisión de diseño explicada en la Sección 8.
2. **CRUD de cartas y requisitos:** cumplido en cuanto a las operaciones y al ciclo Borrador/Publicado. La exportación a PDF y a DOCX —prometida en la propuesta— se sustituyó por la impresión nativa del navegador (`window.print()`) en la vista pública, sin generación de archivos binarios estructurados en el backend.
3. **Autoevaluación docente:** cumplido y ampliado. El sistema entregado va más allá de un formulario fijo: incorpora un *form-builder* configurable por el administrador, ocho tipos de pregunta, *scoring* ponderado por sección, niveles de desempeño automáticos y versionado de formularios que preserva la integridad histórica.
4. **Reportes de cumplimiento:** cumplido en cuanto a los indicadores. La exportación se implementó en formato *XLSX* en el cliente en lugar de *PDF* en el servidor.

9.2. Aprendizajes técnicos

Los principales aprendizajes derivados del desarrollo se enuncian a continuación.

- **Modelar antes de codificar.** El rediseño de junio de 2026 que aplanó los documentos (eliminación de los submodelos Tema, Subtema, Bibliografía, CriterioEvaluacion) enseñó la importancia de contar con la plantilla oficial *antes* de estructurar el esquema. Ante la ausencia de plantilla normativa, un contenido textual libre resultó más flexible y evitó imponer una estructura que la División no había validado.
- **Separar dominio en apps.** Django facilita la separación de dominios en apps independientes con sus propios modelos, vistas, urls y migraciones. Esta disciplina probó ser útil para acotar el impacto de los cambios y para orientar el sistema de pruebas.
- **JWT con *refresh* rotativo.** La combinación *access* corto + *refresh* rotativo mitiga bien el riesgo de filtración de tokens; el precio es que el cliente debe implementar la lógica de *refresh* transparente, lo cual se resolvió con un *interceptor* de *axios*.

- **Testing como red de seguridad.** Las cerca de 4 200 líneas de tests en el backend permitieron refactorizar sin miedo el módulo de autoevaluación en nueve incrementos sucesivos (PR1–PR9), incluyendo cambios profundos como el paso a *scoring* ponderado y el versionado de formularios.
- **Estado del servidor con *TanStack Query*.** Usar una biblioteca dedicada al estado del servidor eliminó una capa entera de código de *fetching* manual y regaló funcionalidades no triviales: *cache*, revalidación, *polling* y *stale-while-revalidate*.

9.3. Trabajo futuro

El sistema entregado es funcional y suficiente para arrancar la operación real de la División, pero se identifican las siguientes líneas de trabajo futuro:

1. **Exportación nativa a PDF.** Incorporar *WeasyPrint* o *ReportLab* en el backend para generar PDFs firmables de cartas temáticas, requisitos de recuperación y resultados de autoevaluación. Esta funcionalidad se prometió en la propuesta y quedó pendiente.
2. **Reset de contraseña autoservicio.** Añadir un flujo de *forgot password* con envío de token único al correo institucional, evitando que el administrador sea el único capaz de restablecer contraseñas.
3. **Integración con directorio institucional.** Si la División cuenta con un LDAP o SSO institucional, integrarlo evitaría mantener contraseñas independientes.
4. **Cobertura de pruebas del frontend.** El frontend actualmente carece de suite de pruebas; añadir Vitest con *React Testing Library* para cubrir al menos los flujos críticos (*login*, *guards* de rol, envío de autoevaluación) elevaría la confianza para futuros cambios.
5. **Estructuración opcional del contenido de cartas.** Si en el futuro la Coordinación de CyAD define una plantilla oficial de carta temática con submodelos (temario por unidad, bibliografía por tipo, criterios de evaluación tabulados), es posible reintroducir la jerarquía sin afectar los documentos existentes mediante una migración adicional.
6. **Notificaciones.** Enviar recordatorios automáticos por correo a profesores que no han publicado cartas o respondido su autoevaluación cerca del cierre del periodo.
7. **Despliegue en el servidor institucional *cyad.azc.uam.mx*.** La puesta en producción está prevista para el trimestre siguiente al reporte, con *Gunicorn* + *Nginx* y respaldos periódicos de PostgreSQL.

En conjunto, el sistema desarrollado sienta una base sólida para la modernización de la administración académica de la División de CyAD y demuestra que un equipo pequeño puede sustituir un flujo manual con *Google Forms* por una solución propia, mantenible y con espacio de crecimiento.

A. Código fuente

Este apéndice presenta *fragmentos curados* del código fuente que ilustran las decisiones de diseño más importantes. El código fuente completo (backend, frontend, tests, scripts y fixtures) se entrega en el archivo `codigo_fuente.zip` de la carpeta de entrega, generado con `git archive HEAD`.

A.1. Estructura del repositorio

```

proyecto_terminal_cyad/
|-- backend/
|   |-- config/           # settings, urls, wsgi
|   |-- core/            # utilidades: pagination, exceptions
|   |-- accounts/        # Usuario, Profesor, JWT
|   |-- catalogos/       # Departamento, Licenciatura, UEA, Periodo
|   |-- documentos/      # CartaTematica, RequisitoRecuperacion
|   |-- autoevaluacion/  # Formulario, Pregunta, Respuesta, Nivel
|   |-- reportes/        # Dashboards y cumplimiento
|   |-- scripts/         # seed_users, seed_demo, create_db
|   |-- tests/           # ~4200 lineas de tests con pytest
|   |-- manage.py
|   |-- requirements.txt
|-- frontend/
|   |-- src/
|   |   |-- api/          # clientes HTTP (auth, documentos, ...)
|   |   |-- auth/         # AuthContext, ProtectedRoute, RoleRoute
|   |   |-- components/ui/ # Button, Alert, Badge, Table, Modal
|   |   |-- layouts/      # AdminLayout, ProfesorLayout
|   |   |-- pages/
|   |   |   |-- admin/    # dashboards y CRUD por catalogo
|   |   |   |-- profesor/ # cartas, requisitos, autoevaluacion
|   |   |   |-- publico/  # vista Explorar y documentos publicados
|   |   |-- App.jsx       # rutas + guards
|   |   |-- main.jsx      # bootstrap React + TanStack Query
|   |-- index.html
|   |-- package.json
|   |-- vite.config.js
|-- docs/
|   |-- reporte/          # este documento
|   |-- manuales/        # instalacion y usuario
|-- README.md

```

A.2. Backend — Modelo de Usuario con roles

Fragmento de `backend/accounts/models.py`: modelo de usuario personalizado con dos roles.

```

class Usuario(AbstractBaseUser, PermissionsMixin):
    class Rol(models.TextChoices):
        ADMIN      = "ADMIN",      "Administrador"
        PROFESOR   = "PROFESOR",    "Profesor"

    email          = models.EmailField(unique=True)
    nombre         = models.CharField(max_length=150)
    rol            = models.CharField(max_length=10, choices=Rol.choices)
    is_active      = models.BooleanField(default=True)
    is_staff       = models.BooleanField(default=False)

    USERNAME_FIELD = "email"
    REQUIRED_FIELDS = ["nombre"]

    objects = UsuarioManager()

```

A.3. Backend — Perfil de Profesor y Periodo

Fragmento de `backend/accounts/models.py` y `backend/catalogos/models.py`: el perfil Profesor sincroniza `Usuario.is_active` con `Profesor.estado` para que desactivar el perfil bloquee el inicio de sesión, y el modelo Periodo garantiza en `save()` la invariante “a lo sumo un periodo activo por recurso”.

```

# backend/accounts/models.py
class Profesor(TimeStampedModel, EstadoActivoModel):
    usuario = models.OneToOneField(
        Usuario, on_delete=models.CASCADE, related_name="perfil_profesor",
        limit_choices_to={"rol": Usuario.Rol.PROFESOR},
    )
    numero_economico = models.CharField(max_length=20, unique=True,
                                         null=True, blank=True)
    nombre_completo = models.CharField(max_length=255)
    correo_institucional = models.EmailField(unique=True)
    departamento = models.ForeignKey(
        "catalogos.Departamento", on_delete=models.SET_NULL,
        null=True, blank=True, related_name="profesores",
    )

    def save(self, *args, **kwargs):
        # Sincroniza el login del Usuario con el estado del perfil.
        super().save(*args, **kwargs)
        if self.usuario_id and self.usuario.is_active != bool(self.estado):
            self.usuario.is_active = bool(self.estado)
            self.usuario.save(update_fields=["is_active"])

```

```
# backend/catalogos/models.py
class Periodo(TimeStampedModel):
    class Recurso(models.TextChoices):
        CARTAS = "CARTAS", "Cartas Temáticas"
        REQUISITOS = "REQUISITOS", "Requisitos de Recuperación"
        AUTOEVALUACION = "AUTOEVALUACION", "Autoevaluación"

    clave = models.CharField(max_length=10, unique=True)
    fecha_inicio = models.DateField()
    fecha_fin = models.DateField()
    activo_cartas = models.BooleanField(default=False)
    activo_requisitos = models.BooleanField(default=False)
    activo_autoevaluacion = models.BooleanField(default=False)
    activo = models.BooleanField(default=False, editable=False) # OR de los tres
    estado = models.BooleanField(default=True)

    @classmethod
    def get_activo(cls, recurso, hoy=None):
        field = {"CARTAS": "activo_cartas",
                 "REQUISITOS": "activo_requisitos",
                 "AUTOEVALUACION": "activo_autoevaluacion"}.get(recurso)
        return cls.objects.filter(**{field: True, "estado": True}).first() if fiel

    def save(self, *args, **kwargs):
        # Invariante: a lo sumo un Periodo con cada flag activo.
        per_resource = ("activo_cartas", "activo_requisitos", "activo_autoevaluaci
        for field in per_resource:
            if getattr(self, field):
                Periodo.objects.exclude(pk=self.pk) \
                    .filter(**{field: True}).update(**{field: False})
        self.activo = any(getattr(self, f) for f in per_resource)
        super().save(*args, **kwargs)
```

A.4. Backend — JWT y throttling

Fragmento de backend/config/settings.py.

```
REST_FRAMEWORK = {
    "DEFAULT_AUTHENTICATION_CLASSES": (
        "rest_framework_simplejwt.authentication.JWTAuthentication",
    ),
    "DEFAULT_PERMISSION_CLASSES": (
        "rest_framework.permissions.IsAuthenticated",
    ),
    "DEFAULT_THROTTLE_CLASSES": (
        "rest_framework.throttling.ScopedRateThrottle",
    ),
```

```

    "DEFAULT_THROTTLE_RATES": {
        "login": "5/min",
    },
    "PAGE_SIZE": 20,
}

SIMPLE_JWT = {
    "ACCESS_TOKEN_LIFETIME": timedelta(minutes=60),
    "REFRESH_TOKEN_LIFETIME": timedelta(days=7),
    "ROTATE_REFRESH_TOKENS": True,
    "AUTH_HEADER_TYPES": ("Bearer",),
}

```

A.5. Backend — Documento academico base

Fragmento de `backend/documentos/models.py`. Clase abstracta con campos comunes y *snapshot* de identidad del profesor.

```

class DocumentoAcademicoBase(TimeStampedModel):
    class Estado(models.TextChoices):
        BORRADOR = "BORRADOR", "Borrador"
        PUBLICADO = "PUBLICADO", "Publicado"

    profesor = models.ForeignKey(
        Profesor, on_delete=models.SET_NULL, null=True, blank=True
    )
    profesor_nombre = models.CharField(max_length=200, blank=True)
    profesor_correo = models.EmailField(blank=True)

    uea = models.ForeignKey(UEA, on_delete=models.PROTECT)
    periodo = models.ForeignKey(Periodo, on_delete=models.PROTECT)
    nombre_grupo = models.CharField(max_length=50)
    id_grupo = models.CharField(max_length=50)
    horario = models.CharField(max_length=200, blank=True)
    modalidad = models.CharField(
        max_length=15,
        choices=(("PRESENCIAL", "Presencial"), ("REMOTO", "Remoto"), ("MIXTO", "Mixto"))
    )
    estado = models.CharField(
        max_length=15,
        choices=Estado.choices,
        default=Estado.BORRADOR,
    )

    class Meta:
        abstract = True

```

A.6. Backend — Modelos del *form-builder*

Fragmento de backend/autoevaluacion/models.py: cinco modelos que definen la estructura editable de un formulario. El ciclo de vida de Formulario (publicar/cerrar/reabrir/despublicar/publicar_revision) se detalla en el listado “Backend — Ciclo de vida de Formulario”.

```
class Formulario(TimeStampedModel):
    class Estado(models.TextChoices):
        BORRADOR = "BORRADOR", "Borrador"
        PUBLICADO = "PUBLICADO", "Publicado"
        CERRADO = "CERRADO", "Cerrado"

    titulo = models.CharField(max_length=255)
    descripcion = models.TextField(blank=True)
    periodo = models.ForeignKey("catalogos.Periodo",
                                on_delete=models.PROTECT,
                                related_name="formularios")
    estado = models.CharField(max_length=10, choices=Estado.choices,
                              default=Estado.BORRADOR)
    version = models.PositiveIntegerField(default=1) # +1 en publicar_revis
    una_respuesta_por_profesor = models.BooleanField(default=True)
    created_by = models.ForeignKey("accounts.Usuario",
                                   on_delete=models.SET_NULL,
                                   null=True, blank=True, related_name="+")
    published_at = models.DateTimeField(null=True, blank=True)
    closed_at = models.DateTimeField(null=True, blank=True)

class Seccion(models.Model):
    formulario = models.ForeignKey(Formulario, on_delete=models.CASCADE,
                                   related_name="secciones")
    titulo = models.CharField(max_length=255)
    descripcion = models.TextField(blank=True)
    orden = models.PositiveSmallIntegerField(default=0)
    # La suma de pesos de las secciones de un formulario debe ser 100 % (±0.01)
    # para poder publicarlo.
    peso = models.DecimalField(max_digits=5, decimal_places=2, default=0)

class Pregunta(models.Model):
    class Tipo(models.TextChoices):
        TEXTO_CORTO = "TEXTO_CORTO", "Texto corto"
        TEXTO_LARGO = "TEXTO_LARGO", "Texto largo"
        OPCION_UNICA = "OPCION_UNICA", "Opción única"
        CASILLAS = "CASILLAS", "Casillas de verificación"
        SI_NO = "SI_NO", "Sí / No"
        ESCALA_LINEAL = "ESCALA_LINEAL", "Escala lineal"
```



```

        LISTA_DESPLEGABLE = "LISTA_DESPLEGABLE", "Lista desplegable"
        CUADRICULA         = "CUADRICULA",         "Cuadrícula de opciones"

# Estas se ignoran en el scoring (respuesta cualitativa abierta).
TIPOS_NO_PUNTABLES = {"TEXTO_CORTO", "TEXTO_LARGO"}

formulario = models.ForeignKey(Formulario, on_delete=models.CASCADE,
                                related_name="preguntas")
seccion    = models.ForeignKey(Seccion, on_delete=models.SET_NULL,
                                null=True, blank=True, related_name="preguntas")
tipo       = models.CharField(max_length=20, choices=Tipo.choices)
texto      = models.TextField()
ayuda      = models.CharField(max_length=500, blank=True)
obligatoria = models.BooleanField(default=True)
orden      = models.PositiveSmallIntegerField(default=0)
# JSON dependiente del tipo:
#   ESCALA_LINEAL -> {min, max, label_min, label_max, puntos_factor}
#   SI_NO         -> {puntos_si, puntos_no}
config       = models.JSONField(default=dict, blank=True)

class OpcionPregunta(models.Model):
    pregunta = models.ForeignKey(Pregunta, on_delete=models.CASCADE,
                                related_name="opciones")
    texto    = models.CharField(max_length=255)
    valor    = models.CharField(max_length=100, blank=True)
    puntos   = models.DecimalField(max_digits=5, decimal_places=2, default=0)
    orden    = models.PositiveSmallIntegerField(default=0)

class FilaCuadricula(models.Model):
    # Las filas de una CUADRICULA. Sus columnas reutilizan OpcionPregunta.
    pregunta = models.ForeignKey(Pregunta, on_delete=models.CASCADE,
                                related_name="filas")
    texto    = models.CharField(max_length=255)
    orden    = models.PositiveSmallIntegerField(default=0)

```

A.7. Backend — Captura de respuestas y baremo

Fragmento de backend/autoevaluacion/models.py: modelos que persisten cada envío del profesor y el baremo (NivelDesempeno) contra el que se evalúa. El algoritmo que rellena `puntaje_obtenido`, `porcentaje`, `nivel_desempeno` y crea un `RespuestaSeccion` por sección se muestra en el listado “Backend — Scoring ponderado de autoevaluación”. El campo `RespuestaSeccion.peso` es un *snapshot* del peso vigente al enviar, precisamente para que el desglose histórico permanezca estable si el administrador reajusta pesos y publica una revisión.

```
class NivelDesempeno(models.Model):
    formulario = models.ForeignKey(Formulario, on_delete=models.CASCADE,
                                   related_name="niveles")
    nombre = models.CharField(max_length=100)
    porcentaje_min = models.DecimalField(max_digits=5, decimal_places=2)
    porcentaje_max = models.DecimalField(max_digits=5, decimal_places=2)
    observacion = models.TextField()
    # Clave para la UI: green | blue | yellow | red | gray
    color = models.CharField(max_length=20, default="gray")
    orden = models.PositiveSmallIntegerField(default=0)

class Respuesta(TimeStampedModel):
    class Estado(models.TextChoices):
        BORRADOR = "BORRADOR", "Borrador"
        ENVIADO = "ENVIADO", "Enviado"

    formulario = models.ForeignKey(Formulario, on_delete=models.PROTECT,
                                   related_name="respuestas")
    profesor = models.ForeignKey("accounts.Profesor",
                                 on_delete=models.PROTECT,
                                 related_name="respuestas_formulario")
    # Copia de Formulario.version al iniciar; permite volver a responder tras
    # publicar_revision sin violar la unicidad compuesta de abajo.
    version_formulario = models.PositiveIntegerField(default=1)
    estado = models.CharField(max_length=10, choices=Estado.choices,
                              default=Estado.BORRADOR)
    enviado_at = models.DateTimeField(null=True, blank=True)
    puntaje_obtenido = models.DecimalField(max_digits=7, decimal_places=2,
                                           null=True, blank=True)
    puntaje_maximo = models.DecimalField(max_digits=7, decimal_places=2,
                                         null=True, blank=True)
    porcentaje = models.DecimalField(max_digits=5, decimal_places=2,
                                     null=True, blank=True)
    nivel_desempeno = models.ForeignKey(NivelDesempeno,
                                        on_delete=models.SET_NULL,
                                        null=True, blank=True,
                                        related_name="respuestas")

    class Meta:
        constraints = [
            models.UniqueConstraint(
                fields=["formulario", "profesor", "version_formulario"],
                name="unique_respuesta_formulario_profesor_version",
            )
        ]
```

```

class RespuestaPregunta(models.Model):
    respuesta = models.ForeignKey(Respuesta,
                                   on_delete=models.CASCADE,
                                   related_name="items")
    pregunta = models.ForeignKey(Pregunta,
                                   on_delete=models.PROTECT,
                                   related_name="respuestas_recibidas")
    valor_texto = models.TextField(blank=True)
    opciones_seleccionadas = models.ManyToManyField(OpcionPregunta, blank=True,
                                                    related_name="selecciones")

    class Meta:
        unique_together = [("respuesta", "pregunta")]

class RespuestaSeccion(models.Model):
    # Snapshot por sección al enviar: preserva el peso vigente para el desglose.
    respuesta = models.ForeignKey(Respuesta, on_delete=models.CASCADE,
                                   related_name="secciones_resultado")
    seccion = models.ForeignKey(Seccion, on_delete=models.PROTECT,
                                   related_name="resultados")
    peso = models.DecimalField(max_digits=5, decimal_places=2)
    puntaje_obtenido = models.DecimalField(max_digits=7, decimal_places=2)
    puntaje_maximo = models.DecimalField(max_digits=7, decimal_places=2)
    porcentaje = models.DecimalField(max_digits=5, decimal_places=2,
                                      default=0)

    class Meta:
        unique_together = [("respuesta", "seccion")]

```

A.8. Backend — Scoring ponderado de autoevaluación

Fragmento simplificado de `backend/autoevaluacion/views.py`: cálculo del puntaje ponderado por sección, con manejo por tipo de pregunta y *snapshot* del peso en `RespuestaSeccion`.

```

def _calcular_puntaje(respuesta):
    resultado_secciones = []
    item_by_pregunta = {r.pregunta_id: r for r in respuesta.items.all()}
    for seccion in respuesta.formulario.secciones.all():
        sec_obt, sec_max = Decimal("0"), Decimal("0")
        for pregunta in seccion.preguntas.all():
            if pregunta.tipo in Pregunta.TIPOS_NO_PUNTABLES:
                continue
            item = item_by_pregunta.get(pregunta.id)
            if pregunta.tipo == Pregunta.Tipo.CUADRICULA:

```

```

        opts, filas = list(pregunta.opciones.all()), list(pregunta.filas.all())
        if opts and filas:
            sec_max += max(o.puntos for o in opts) * len(filas)
        if item:
            sec_obt += sum(c.opcion.puntos for c in item.celdas.all())
    elif pregunta.tipo == Pregunta.Tipo.ESCALA_LINEAL:
        cfg = pregunta.config or {}
        factor = Decimal(str(cfg.get("puntos_factor", 1)))
        sec_max += Decimal(str(cfg.get("max", 5))) * factor
        if item and item.valor_texto:
            sec_obt += Decimal(item.valor_texto) * factor
    # ... SI_NO, CASILLAS, OPCION_UNICA / LISTA_DESPLEGABLE ...
    porcentaje = (sec_obt / sec_max * 100).quantize(Decimal("0.01")) \
        if sec_max > 0 else Decimal("0")
    resultado_secciones.append({
        "seccion": seccion, "obtenido": sec_obt, "maximo": sec_max,
        "porcentaje": porcentaje, "peso": seccion.peso,
    })

porcentaje_global = sum(
    s["porcentaje"] * s["peso"] / Decimal("100") for s in resultado_secciones
).quantize(Decimal("0.01"))

with transaction.atomic():
    respuesta.porcentaje = porcentaje_global
    respuesta.nivel_desempeno = NivelDesempeno.objects.filter(
        formulario=respuesta.formulario,
        porcentaje_min__lte=porcentaje_global,
        porcentaje_max__gte=porcentaje_global,
    ).first()
    respuesta.save()
    for s in resultado_secciones:
        RespuestaSeccion.objects.update_or_create(
            respuesta=respuesta, seccion=s["seccion"],
            defaults={
                "puntaje": s["obtenido"],
                "puntaje_maximo": s["maximo"],
                "porcentaje": s["porcentaje"],
                "peso": s["peso"], # snapshot para desglose histor
            },
        )

```

A.9. Backend — Permisos por rol

Fragmento del sistema de permisos que garantiza que cada profesor solo edite sus documentos.

```
class IsOwnerProfesorOrAdminReadOnly(permissions.BasePermission):
    def has_permission(self, request, view):
        return request.user.is_authenticated

    def has_object_permission(self, request, view, obj):
        if request.user.rol == Usuario.Rol.ADMIN:
            return request.method in permissions.SAFE_METHODS
        return obj.profesor and obj.profesor.usuario_id == request.user.id
```

A.10. Frontend — Cliente HTTP con refresh automatico de JWT

Fragmento de frontend/src/api/client.js.

```
const client = axios.create({
  baseURL: import.meta.env.VITE_API_URL,
});

client.interceptors.request.use((config) => {
  const token = sessionStorage.getAccessToken();
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

let refreshing = null;

client.interceptors.response.use(
  (r) => r,
  async (error) => {
    const { config, response } = error;
    if (response?.status === 401 && !config.__retried) {
      config.__retried = true;
      refreshing = refreshing ?? refreshAccessToken();
      const newToken = await refreshing.finally(() => (refreshing = null));
      if (newToken) {
        config.headers.Authorization = `Bearer ${newToken}`;
        return client(config);
      }
    }
    return Promise.reject(error);
  }
);
```

A.11. Frontend — Guard de rutas por rol

Fragmento de frontend/src/auth/RoleRoute.jsx.

```

export function RoleRoute({ role, children }) {
  const { user, isLoading } = useAuth();
  if (isLoading) return <Loading />;
  if (!user) return <Navigate to="/login" replace />;
  if (user.rol !== role) return <Navigate to="/" replace />;
  return children;
}

export const AdminRoute = ({ children }) =>
  <RoleRoute role="ADMIN" >{children}</RoleRoute>;
export const ProfesorRoute = ({ children }) =>
  <RoleRoute role="PROFESOR" >{children}</RoleRoute>;

```

A.12. Frontend — Renderizador dinámico de preguntas

Fragmento de frontend/src/pages/profesor/autoevaluacion/AutoevaluacionFormPage.jsx que despacha por tipo de pregunta.

```

function PreguntaRenderer({ pregunta, value, onChange }) {
  switch (pregunta.tipo) {
    case "TEXTO_CORTO":      return <TextInput  {...{value, onChange}} />;
    case "TEXTO_LARGO":      return <TextArea   {...{value, onChange}} />;
    case "OPCION_UNICA":     return <RadioGroup opciones={pregunta.opciones} {...
    case "CASILLAS":         return <CheckGroup opciones={pregunta.opciones} {...
    case "SI_NO":            return <YesNo       {...{value, onChange}} />;
    case "ESCALA_LINEAL":    return <LinearScale config={pregunta.config} {...{va
    case "LISTA_DESPLEGABLE": return <Select      opciones={pregunta.opciones} {...
    case "CUADRICULA":       return <Grid        config={pregunta.config} filas={p
    default:                 return <UnknownType tipo={pregunta.tipo} />;
  }
}

```

A.13. Backend — Comando de importación CSV para UEA

Comando de gestión python manage.py cargar_catalogos_csv para carga masiva idempotente. Fragmento simplificado.

```

class Command(BaseCommand):
    help = "Carga catalogos (Areas, Licenciaturas, Posgrados, UEAs) desde CSV"

    def add_arguments(self, parser):
        parser.add_argument("--ueas", type=str, help="Path al CSV de UEAs")

    def handle(self, *args, **opts):
        ruta = opts["ueas"]

```

```

with open(ruta, newline="", encoding="utf-8") as fh:
    for fila in csv.DictReader(fh):
        UEA.objects.update_or_create(
            clave=fila["clave"],
            licenciatura=Licenciatura.objects.get(clave=fila["licenciatura"],
            defaults={
                "nombre": fila["nombre"],
                "trimestre": int(fila["trimestre"]),
                "creditos": int(fila["creditos"]),
                "tipo": fila["tipo"],
            },
        )

```

A.14. Backend — Constraint XOR y unicidad parcial de UEA

Fragmento de `backend/catalogos/models.py`: reglas de integridad declarativas que fuerzan que una UEA pertenezca a licenciatura o a posgrado (nunca a ambos ni a ninguno), y que la clave sea unica dentro de cada programa.

```

class Meta:
    constraints = [
        CheckConstraint(
            condition=(
                (Q(licenciatura__isnull=False) & Q(posgrado__isnull=True))
                | (Q(licenciatura__isnull=True) & Q(posgrado__isnull=False))
            ),
            name="uea_xor_licenciatura_posgrado",
        ),
        UniqueConstraint(
            fields=["clave", "licenciatura"],
            condition=Q(licenciatura__isnull=False),
            name="uea_clave_unica_por_licenciatura",
        ),
        UniqueConstraint(
            fields=["clave", "posgrado"],
            condition=Q(posgrado__isnull=False),
            name="uea_clave_unica_por_posgrado",
        ),
    ]

```

A.15. Backend — Ciclo de vida de Formulario

Fragmento de `backend/autoevaluacion/models.py`: validacion de estructura y transiciones de la maquina de estados BORRADOR ↔ PUBLICADO ↔ CERRADO.

```

def _validar_estructura_para_publicar(self):
    secciones = list(self.secciones.all())

```

```

        if not secciones:
            raise ValueError("Debe definir al menos una seccion antes de publicar.")
        suma = sum(s.peso for s in secciones)
        if abs(suma - Decimal("100")) > Decimal("0.01"):
            raise ValueError(f"La suma de pesos debe ser 100% (actual: {suma}%).")
        if self.preguntas.filter(seccion__isnull=True).exists():
            raise ValueError("Hay preguntas sin seccion asignada.")

def publicar(self):
    self._validar_estructura_para_publicar()
    if self.estado == self.Estado.CERRADO and self.respuestas.exists():
        self.version += 1      # publicar_revision incrementa version
    self.estado = self.Estado.PUBLICADO
    self.published_at = timezone.now()
    self.save()

def cerrar(self):
    self.estado = self.Estado.CERRADO
    self.closed_at = timezone.now()
    self.save()

def reabrir(self):
    self.estado = self.Estado.PUBLICADO
    self.save()

def despublicar(self):
    # Vuelve a BORRADOR sin cambiar version ni borrar respuestas.
    self.estado = self.Estado.BORRADOR
    self.save()

def publicar_revision(self):
    # CERRADO -> PUBLICADO incrementando version.
    self._validar_estructura_para_publicar()
    self.version += 1
    self.estado = self.Estado.PUBLICADO
    self.published_at = timezone.now()
    self.save()

```

A.16. Backend — Login con distincion de cuenta desactivada

Fragmento de backend/accounts/serializers.py. El *serializer* personalizado distingue *credenciales invalidas* de *cuenta desactivada* devolviendo un codigo `account_disabled` sin filtrar la existencia del correo.

```

class CustomTokenObtainPairSerializer(TokenObtainPairSerializer):
    def validate(self, attrs):
        email = (attrs.get(self.username_field) or "").strip().lower()

```



```

password = attrs.get("password") or ""
usuario = Usuario.objects.filter(email__iexact=email).first()
if usuario and usuario.check_password(password) and not usuario.is_active:
    raise AuthenticationFailed(
        {"code": "account_disabled",
         "detail": "Tu cuenta se encuentra desactivada. "
                  "Contacta al administrador para reactivarla."},
        code="account_disabled",
    )
data = super().validate(attrs) # credenciales OK, usuario activo
u = self.user
data["user"] = {"id": u.id, "email": u.email,
                 "nombre": u.nombre, "rol": u.rol}

return data

```

A.17. Frontend — Generacion de XLSX por periodo

Fragmento simplificado de frontend/src/pages/admin/reportes/Reportes Page.jsx: descarga en el navegador con *SheetJS*, agrupando las filas por periodo en hojas separadas del mismo libro.

```

async function descargarDocumento(publicPath, fetcher) {
  const params = { estado: "PUBLICADO", page_size: 500 }
  if (periodoId) params.periodo = periodoId
  if (deptoId)   params.profesor__departamento = deptoId

  const res = await fetcher(params)
  const rows = res.data?.results ?? res.data ?? []

  const porPeriodo = new Map()
  for (const row of rows) {
    const clave = row.periodo_clave ?? "Sin periodo"
    if (!porPeriodo.has(clave)) porPeriodo.set(clave, [])
    porPeriodo.get(clave).push(row)
  }

  const wb = XLSX.utils.book_new()
  if (porPeriodo.size === 0) {
    XLSX.utils.book_append_sheet(wb, XLSX.utils.json_to_sheet([]), "Sin datos")
  } else {
    for (const [clave, groupRows] of porPeriodo) {
      const ws = XLSX.utils.json_to_sheet(
        groupRows.map((r) => rowToExcel(r, publicPath))
      )
      XLSX.utils.book_append_sheet(wb, ws, sheetName(clave))
    }
  }
}

```

```
XLSX.writeFile(wb, `${publicPath}-${Date.now()}.xlsx`)
}
```

A.18. Nota sobre el listado completo

Los archivos aquí incluidos son una selección representativa de los fragmentos más ilustrativos. El código fuente completo (backend, frontend, *tests*, scripts y fixtures) se entrega en el archivo `codigo_fuente.zip`. Para regenerarlo desde el repositorio se ejecuta desde la raíz del proyecto:

```
git archive HEAD -o docs/reporte/entregables/codigo_
fuente.zip
```

Adicionalmente, la documentación viva de la API se genera automáticamente vía *drf-spectacular* y se sirve en `/api/docs/` (interfaz Swagger UI) y `/api/schema/` (esquema OpenAPI en YAML).

B. Entregables del plan de trabajo

La Tabla 5 replica el calendario de actividades comprometido en la propuesta (198 horas en total para el Trimestre 2026-Primavera) con el estado real de cada entregable al cierre del proyecto.

Tabla 5. Entregables prometidos en la propuesta y su estado real.

No.	Actividad	Hrs.	Estado y evidencia
1	Diseño e implementación de la base de datos	20	Completo. 21 migraciones, seis apps, PostgreSQL 16. Diagrama ER en Figura 3.
2	Diseño de las interfaces por módulo	10	Completo. UI implementada en React 19 + Tailwind, con componentes reutilizables (<i>Button, Alert, Badge, Table, Modal, FormField, etc</i>). Capturas en Sección 7.
3	Desarrollo del módulo de autenticación	25	Completo. <code>accounts/</code> + 491 líneas de <i>tests</i> (<code>tests/test_auth.py</code>). Login JWT con <i>throttling</i> .
4	Desarrollo del módulo de cartas temáticas	25	Completo. <code>documentos/</code> (parte <i>CartaTematica</i>) + <i>tests</i> en <code>tests/test_documentos.py</code> . Sin exportación PDF/DOCX (véase Sección 8).

No.	Actividad	Hrs.	Estado y evidencia
5	Desarrollo del módulo de requisitos de recuperación	25	Completo. documentos/ (parte RequisitoRecuperacion). Mismo alcance/limitaciones que cartas.
6	Desarrollo del módulo de autoevaluación docente	25	Ampliado. autoevaluacion/ con form-builder, 8 tipos de pregunta, scoring ponderado y versionado (9 incrementos PR1-PR9). 2 044 líneas de tests.
7	Desarrollo del módulo de generación de reportes	25	Completo. reportes/ + 390 líneas de tests. Export XLSX en cliente (no PDF).
8	Integración del sistema con la base de datos	10	Completo. ORM operativo, fixtures, comando cargar_catalogos_csv y scripts seed_users / seed_demo.
9	Prueba y corrección de errores	8	Completo. ≈4 186 líneas de tests + smoke test end-to-end. Historial git con fixes rastreables (por ejemplo fix(seguridad), fix(autoevaluacion), fix(catalogos)).
10	Documentación para usuarios	10	Completo. docs/manuales/usuario.tex y docs/manuales/instalacion.tex.
11	Redacción del reporte final del proyecto	15	Completo. Este documento (docs/reporte/main.pdf) con nueve secciones, dos anexos, índices y bibliografía.
<i>Entregables adicionales no contemplados originalmente:</i>			
E1	Vista pública de exploración (cascada periodo→programa→UEA)	—	Entregado (extra). Beneficia a estudiantes al elegir carga.
E2	Catálogos administrables completos + importación CSV	—	Entregado (extra). Departamentos, licenciaturas, posgrados, áreas, UEA, periodos.
E3	Suite de pruebas de ~4 200 líneas en backend	—	Entregado (extra). Ver backend/tests/.
E4	Documentación OpenAPI/Swagger auto-generada	—	Entregado (extra). Vía drf-spectacular en /api/schema/ y /api/docs/.

Adicionalmente, la carpeta digital de entrega a la Coordinación de Estudios contiene:

- `reporte_final.pdf` — compilación de este documento, sin restricciones.

- `codigo_fuente.zip` — generado con `git archive HEAD`, descomprimible sin contraseña.
- `manual_usuario.pdf` y `manual_instalacion.pdf`.
- `cobertura/` — reporte HTML de cobertura de tests generado con `pytest -cov`.

Referencias

- [1] H. P. Leyva López, M. G. Pérez Vera, and S. M. Pérez Vera, "Google forms en la evaluación diagnóstica como apoyo en las actividades docentes. caso con estudiantes de la licenciatura en turismo," *RIDE. Revista Iberoamericana para la Investigación y el Desarrollo Educativo*, vol. 9, no. 17, pp. 84–111, 2018.
- [2] J. M. A. Echeverría, "La autoevaluación docente de aula: un camino para mejorar la práctica educativa," *Revista Electrónica Diálogos Educativos. REDE*, vol. 11, no. 22, pp. 183–196, 2011.
- [3] A. Cuautle Vasquez, "Sistema para la configuración de formularios y captura de los resultados de la evaluación de atributos de egreso en un proceso de mejora continua," Proyecto terminal, División de Ciencias Básicas e Ingeniería, Universidad Autónoma Metropolitana Azcapotzalco, México, 2022.
- [4] G. Callisaya and A. Emerson, "Sistema académico de gestión escolar trimestralizado," *Computer*, 2024. [Online]. Available: <https://repositorio.upea.bo/jspui/handle/123456789/1062>
- [5] E. M. Rojas, M. R. Ramírez, H. B. R. Moreno, M. d. C. S. Soto, C. O. Millán, Nora d., and L. M. C. Suarez, "Sistema de gestión académica a través del desarrollo de modelo-vista-controlador," *Computer*, vol. 20, no. 9, pp. 1083–1093, 01 2019. [Online]. Available: <https://www.proquest.com/scholarly-journals/sistema-de-gesti3n-a-ca3lmica-trav3ls-del/docview/2195127267/se-2>
- [6] J. C. González *et al.*, "Sistema de gestión administrativa para la escuela secundaria n° 44 "jorge luis borges"," B.S. thesis, Facultad de Tecnología y Ciencias Aplicadas. Departamento de Informática., 2017.
- [7] M. G. P. Martínez, "La digitalización de las instituciones de educación superior en el salvador: Retos y oportunidades," *Perspectiva Científica*, vol. 1, no. 2, pp. 27–32, 2024.
- [8] G. E. Krasner and S. T. Pope, "A cookbook for using the model-view-controller user interface paradigm in smalltalk-80," *Journal of Object-Oriented Programming*, vol. 1, no. 3, pp. 26–49, 1988.
- [9] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, University of California, Irvine, 2000. [Online]. Available: <https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm>
- [10] Django Software Foundation, "Django documentation," 2024. [Online]. Available: <https://docs.djangoproject.com/en/5.1/>
- [11] T. Christie and Django REST Framework contributors, "Django REST Framework documentation," 2024. [Online]. Available: <https://www.django-rest-framework.org/>

- [12] PostgreSQL Global Development Group, "PostgreSQL 16 documentation," 2024. [Online]. Available: <https://www.postgresql.org/docs/16/>
- [13] Meta Open Source, "React documentation," 2024. [Online]. Available: <https://react.dev/>
- [14] Vite contributors, "Vite: Next generation frontend tooling," 2024. [Online]. Available: <https://vitejs.dev/>
- [15] Tailwind Labs, "Tailwind CSS: A utility-first css framework," 2024. [Online]. Available: <https://tailwindcss.com/docs>
- [16] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, Internet Engineering Task Force (IETF), 2015. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc7519>
- [17] E. Sánchez and R. Villalobos, "Autenticación y autorización basadas en tokens para aplicaciones web modernas," *Research in Computing Science*, vol. 148, no. 5, pp. 103–112, 2019.
- [18] OWASP Foundation, "OWASP Top 10: The ten most critical web application security risks," 2021. [Online]. Available: <https://owasp.org/Top10/>

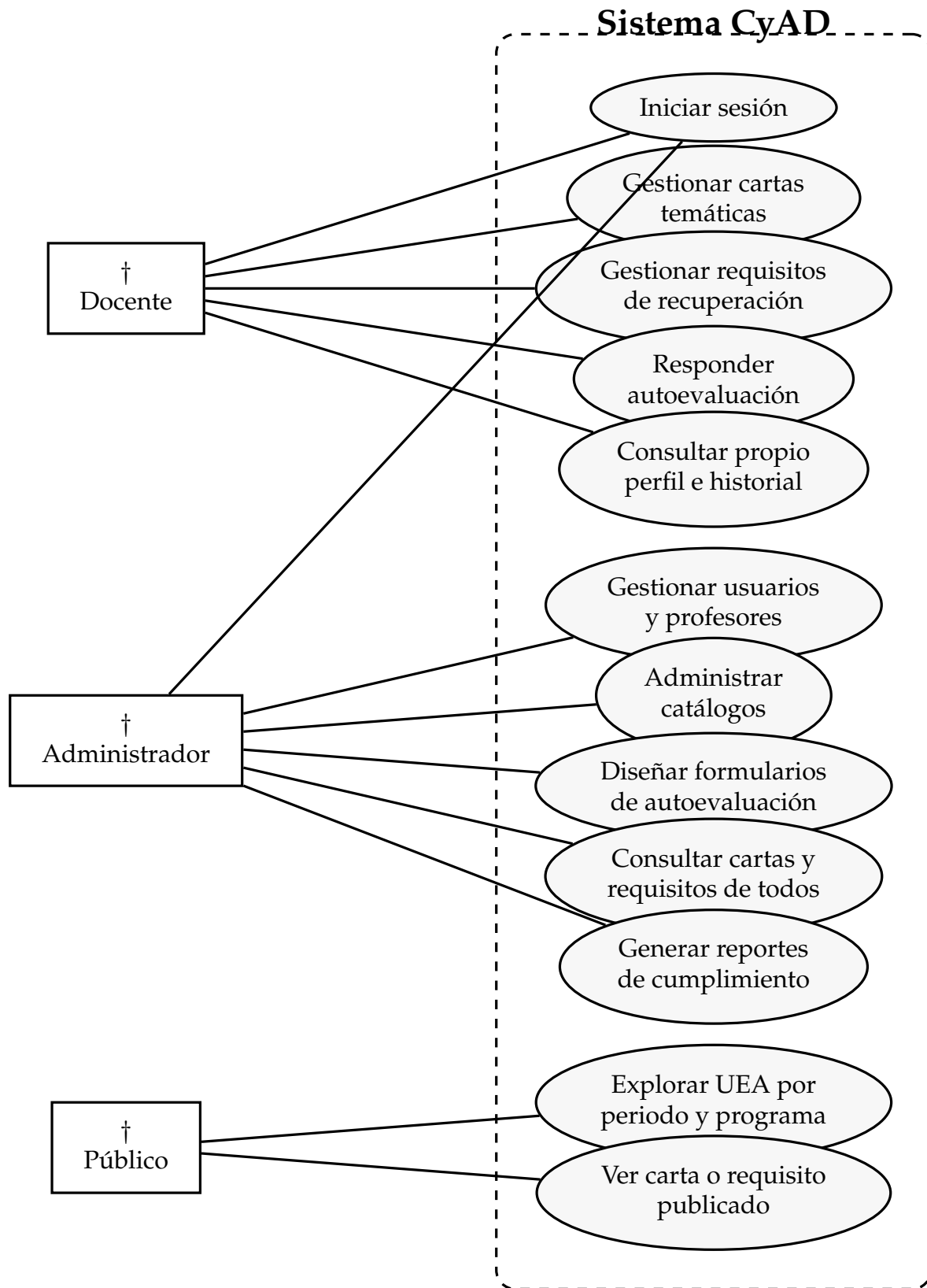


Figura 2. Diagrama de casos de uso del sistema.

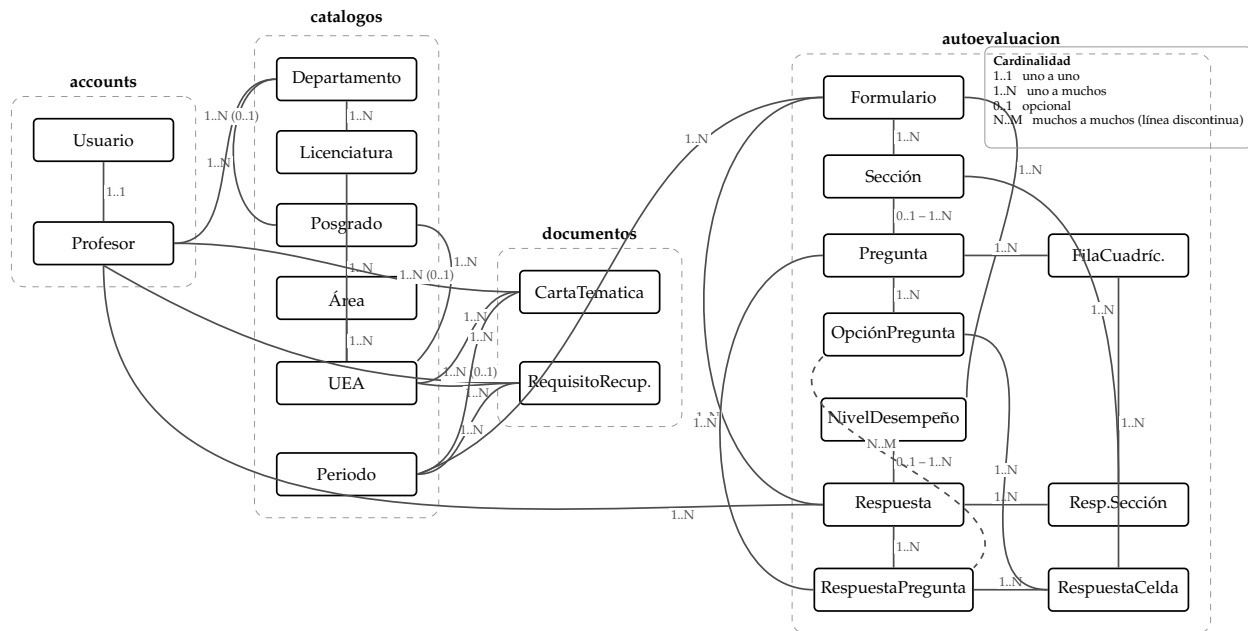
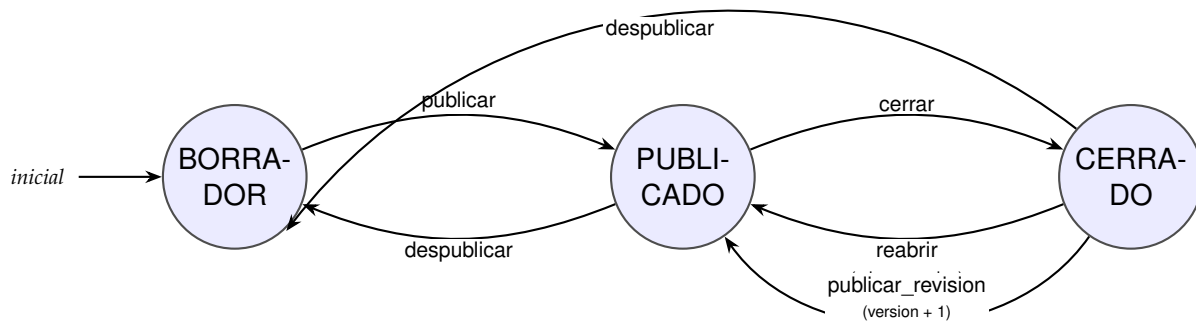


Figura 3. Diagrama Entidad-Relación del sistema (agrupado por app Django).



Accion transversal duplicar: clona la estructura completa a otro periodo (sin arrastrar respuestas) desde cualquier estado.

Figura 4. Máquina de estados del Formulario de autoevaluación. Las transiciones que preservan la versión se dibujan a la izquierda entre **PUBLICADO** y **CERRADO**; la revisión con incremento de versión (**publicar_revision**) se traza como arista externa.

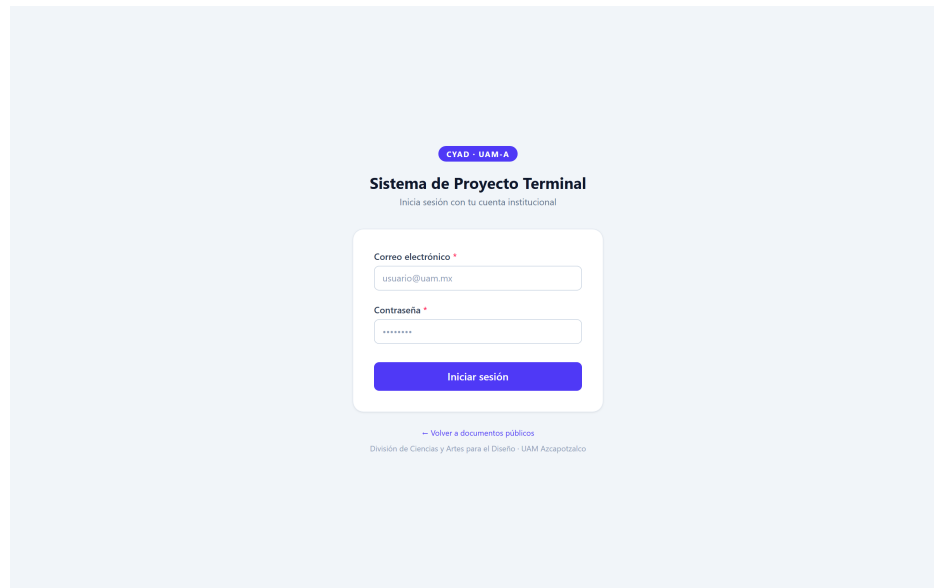


Figura 5. Pantalla de inicio de sesión.

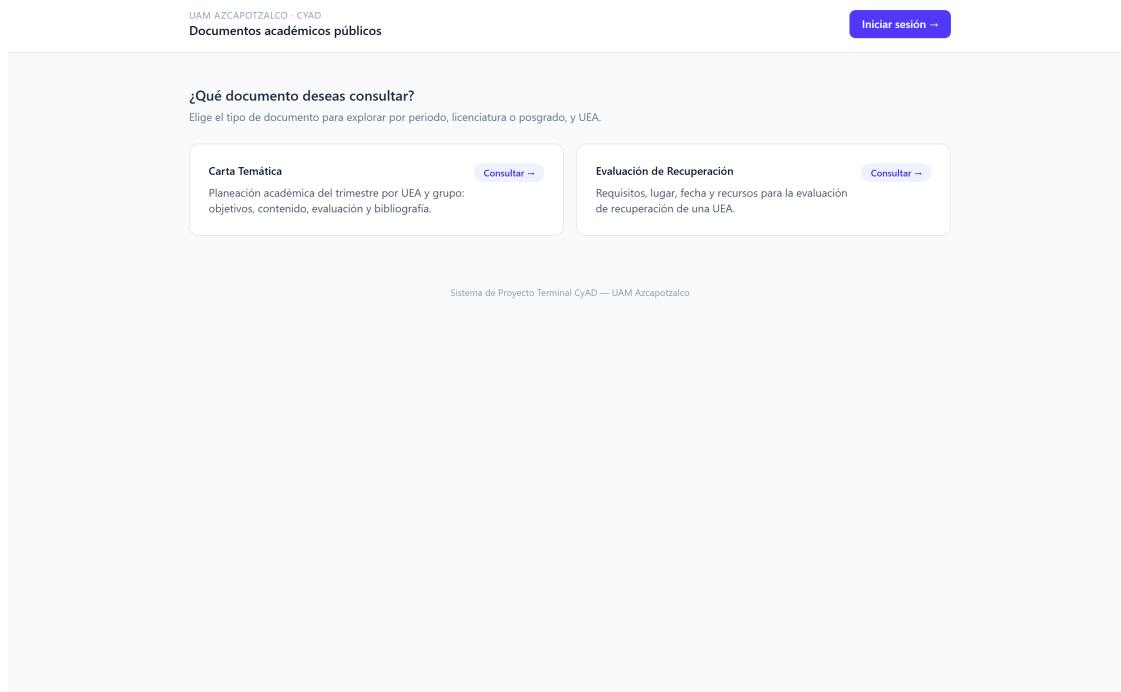


Figura 6. *Dashboard* del profesor: tarjetas de resumen y listas agrupadas por periodo.

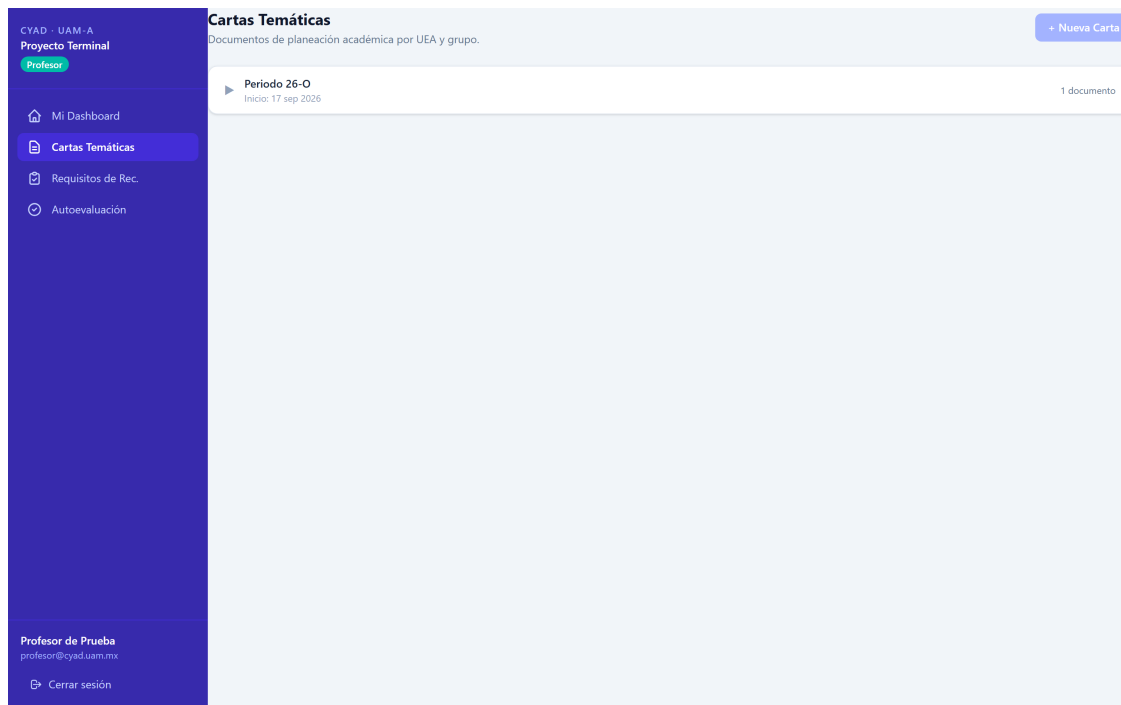


Figura 7. Lista de cartas temáticas del profesor.

Figura 8. Formulario de creación/edición de carta temática.

The screenshot shows the 'Nuevo Requisito de Recuperación' (New Recovery Requirement) form. The left sidebar contains the user's profile (Profesor) and navigation links: Mi Dashboard, Cartas Temáticas, Requisitos de Rec. (selected), and Autoevaluación. The main content area is divided into sections: 'Selección de UEA' with a search bar and instructions; 'Información del grupo' with fields for Periodo (a warning message is displayed), Modalidad de la UEA (dropdown), Nombre del grupo, ID del grupo, and Horario; and 'Detalles de la evaluación' with fields for Lugar, Texto libre, Duración aproximada, and Fecha y hora.

Figura 9. Formulario de requisitos de recuperación.

The screenshot shows the 'Autoevaluación' (Self-evaluation) form. The left sidebar is identical to Figure 9, with 'Autoevaluación' selected in the navigation menu. The main content area displays the title 'Autoevaluación' and a subtitle 'Formularios de autoevaluación docente agrupados por periodo.' Below this, there is a section for 'Periodo 26-P' with the start date 'Inicio: 06 may 2026' and a '1 documento' indicator. The rest of the form area is currently blank, indicating dynamic rendering of questions.

Figura 10. Formulario de autoevaluación con renderizado dinámico de preguntas.

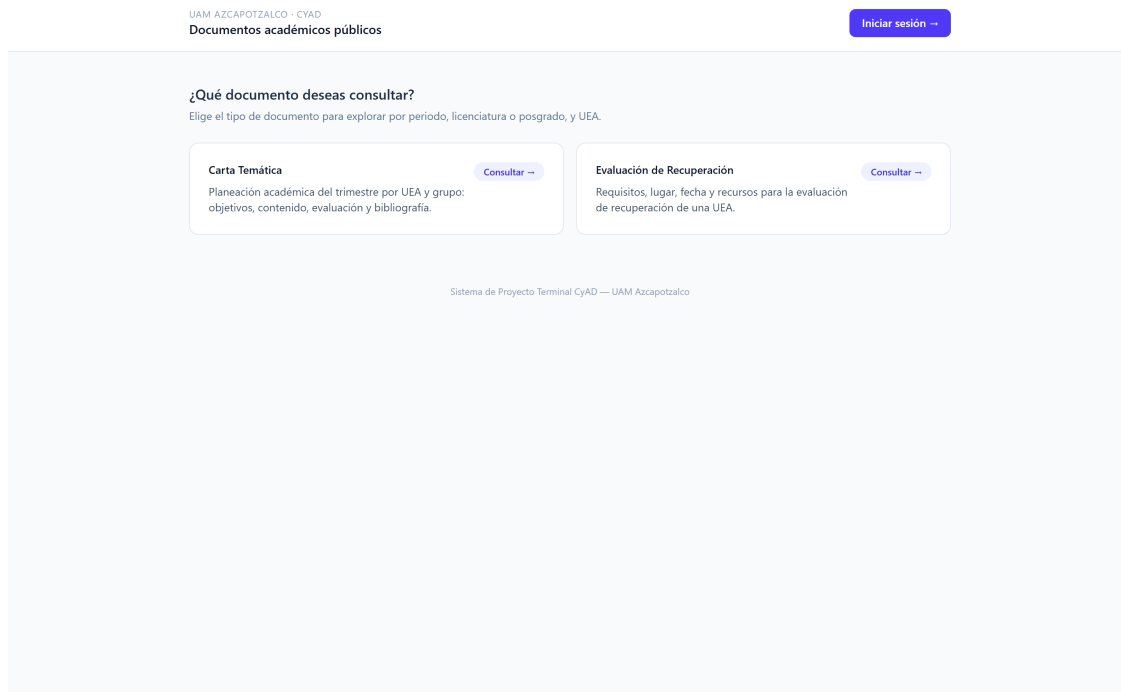


Figura 11. *Dashboard* del administrador: KPIs globales y cumplimiento por departamento.

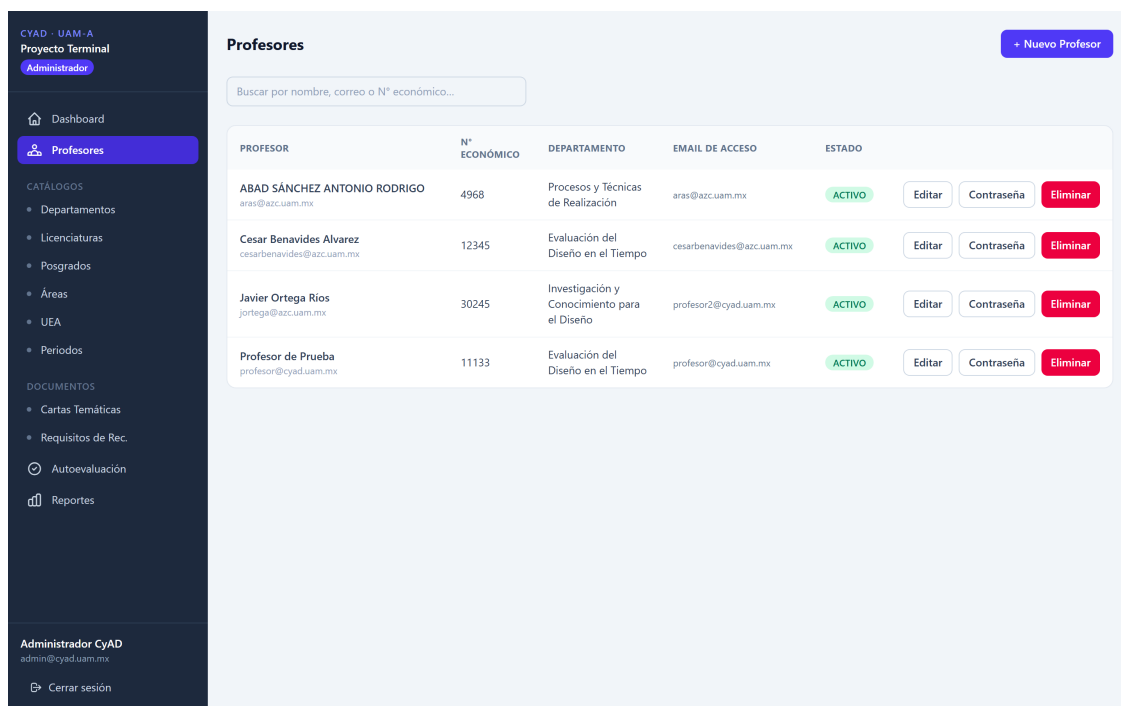


Figura 12. Gestión de profesores: crear, editar, restablecer contraseña.

Autoevaluación
Formularios de evaluación docente.

+ Nuevo Formulario

FORMULARIO	ESTADO	PREGUNTAS	RESPUESTAS	
Autoevaluación Docente 26-I 26-I	BORRADOR	6	0	Ver Editar Duplicar Eliminar
Autoevaluación Docente 26-P completo 26-P	BORRADOR	41	0	Ver Editar Duplicar Eliminar
Autoevaluación Docente 26-O 26-O	PUBLICADO	24	1	Ver Editar Duplicar Eliminar
Autoevaluación Docente 26-P 26-P	PUBLICADO	24	3	Ver Editar Duplicar Eliminar

Figura 13. Constructor de formularios de autoevaluación (*form-builder*).

Reportes
Descarga de documentos.

Descargar Excel
Genera un Excel con los documentos publicados del tipo seleccionado. Si eliges "Todos los periodos", el archivo tendrá una hoja por cada periodo con datos. Filtra opcionalmente por departamento del profesor.

Tipo de documento
Cartas Temáticas

Periodo
Todos los periodos

Departamento
— Todos —

Descargar Excel

Figura 14. Panel de reportes con descarga en XLSX.

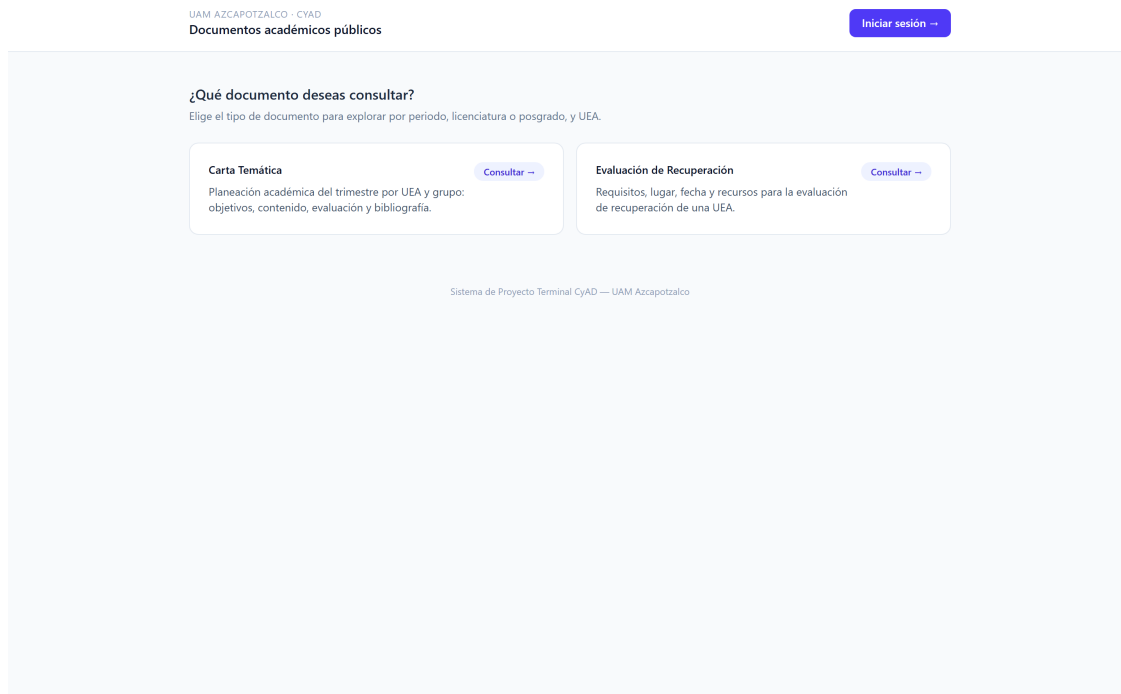


Figura 15. Vista pública *Explorar*: cascada periodo→programa→UEA con acordeón de documentos.

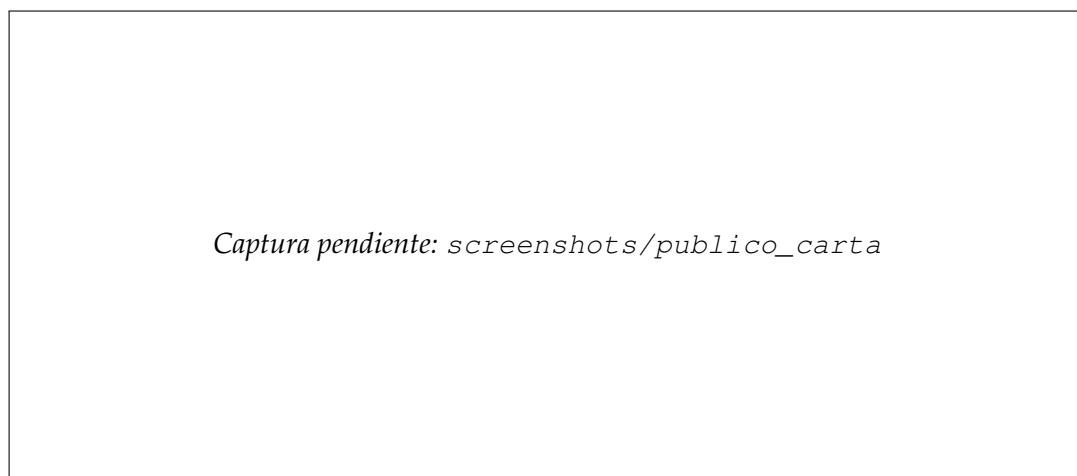


Figura 16. Vista pública de una carta temática publicada, con formato imprimible.